

How to use docker deploying project

Hoàng Đình Thịnh

Ngày 22 tháng 7 năm 2024

Saigon University, Vietnam

thinhkg2002@gmail.com

Tóm tắt nội dung

Docker là một nền tảng phần mềm dùng để xây dựng, đóng gói, và triển khai ứng dụng bằng cách sử dụng containers. Containers là một cách để đóng gói và cô lập các ứng dụng cũng như môi trường chạy của chúng, giúp chúng chạy một cách độc lập và nhất quán trên bất kỳ môi trường nào. Docker cung cấp một cách tiện lợi và linh hoạt để triển khai ứng dụng, giảm thiểu sự phụ thuộc vào môi trường và tối ưu hóa việc sử dụng tài nguyên.

Từ khóa: Quản lý image, triển khai phần mềm, Docker.

1 Giới thiệu

1.1 Containers, Images, Docker File và Docker Compose

- **Containers:** Là một môi trường chứa đựng ứng dụng cùng với tất cả các thành phần cần thiết để chạy ứng dụng đó, bao gồm mã nguồn, thư viện và các tài nguyên khác. Containers là một cách tiêu biểu để đảm bảo tính nhất quán và di động của ứng dụng.
- **Images:** Là một mẫu hoặc một bản ghi chỉ định cách một container cụ thể nên được đóng gói. Images được sử dụng để tạo ra các containers. Chúng bao gồm một tập hợp các lớp (layers) chỉ định cách phải cấu hình một container và ứng dụng cụ thể.
- **Docker File:** Dockerfile là một tập tin văn bản chứa các lệnh để xây dựng một Docker image. Mỗi lệnh trong Dockerfile sẽ tạo ra một lớp (layer) mới trong image, giúp tối ưu hóa việc sử dụng lại các lớp chung giữa các images khác nhau.
- **Docker Compose:** Là một công cụ cho phép định nghĩa và quản lý các ứng dụng đa containers. Bằng cách sử dụng một tệp cấu hình YAML duy nhất, Docker Compose giúp tự động hóa việc tạo ra và quản lý các containers, mạng và volumes cần thiết cho một ứng dụng hoặc dịch vụ.

Docker là một nền tảng mã nguồn mở giúp tự động hóa quá trình triển khai ứng dụng dưới dạng các container. Với Docker, mỗi thành viên trong dự án có thể làm việc độc lập trên các phiên bản container của ứng dụng mà không cần lo lắng về sự khác biệt trong môi trường phát triển. Docker cung cấp các tính năng như xây dựng (build), đẩy (push), và kéo (pull) images, cho phép các nhà phát triển quản lý và chia sẻ môi trường phát triển một cách hiệu quả.

Với sự phổ biến của các kiến trúc microservices và yêu cầu về sự linh hoạt trong triển khai phần mềm, Docker đã trở thành một công cụ không thể thiếu cho các nhà phát triển phần mềm ở mọi cấp độ, từ các dự án cá nhân đến các tổ chức lớn. Ngày nay, với nhu cầu lớn về việc đóng gói và triển khai ứng dụng một cách nhanh chóng và nhất quán, có rất nhiều dịch vụ cung cấp các nền tảng container hóa dựa trên Docker cho các dự án phát triển phần mềm.

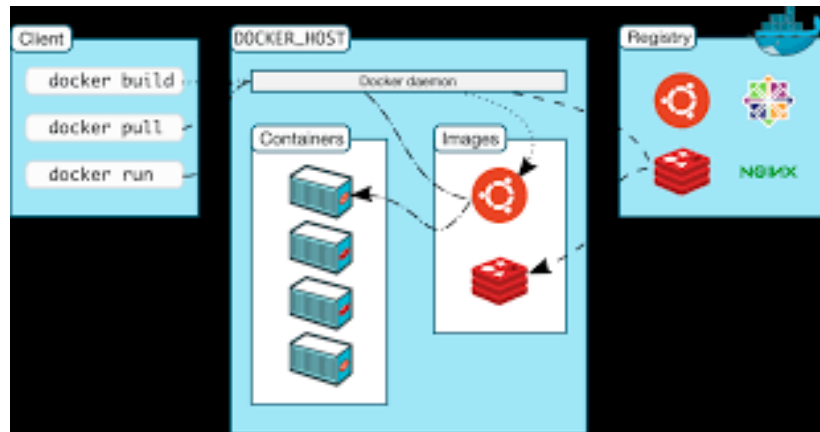


Hình 1: hình ảnh demo

Trong bài viết này, chúng tôi sẽ mang đến cho người đọc một cái nhìn tổng quát và rõ ràng về việc quản lý và triển khai ứng dụng bằng Docker. Phần 2 của bài viết sẽ viết về kiến trúc của Docker, từ Docker Engine, Docker Images đến Docker Containers. Tất tần tật các câu lệnh xử lý trên Docker sẽ được đề cập ở phần 3. Và phần 4 sẽ thực nghiệm trên một số dịch vụ Docker phổ biến như Docker Hub và Kubernetes.

2 Kiến Trúc của Docker

Kiến trúc của Docker được thiết kế để linh hoạt, phân tán và bảo đảm tính nhất quán trong quản lý các ứng dụng được đóng gói. Một số điểm cơ bản về kiến trúc của Docker có thể bao gồm: hệ thống container, quản lý hình ảnh, mạng lưới và lưu trữ, quản lý người dùng và quyền hạn, cũng như cơ chế giám sát và điều khiển.



Hình 2: Kiến trúc của Docker

- **Client và DOCKER_HOST:**

- Client giao tiếp với Docker thông qua các lệnh như `docker build`, `docker pull`, và `docker run`. Điều này tương tự như việc sử dụng các lệnh Git như `git commit`, `git push`, và `git pull` để tương tác với hệ thống DVCS.
- DOCKER_HOST chứa Docker daemon, Containers, và Images. Điều này tương tự như cách mỗi người dùng có bản sao đầy đủ của toàn bộ lịch sử mã nguồn trên máy của mình khi sử dụng Git.

- **Docker daemon:**

- Điều khiển việc xây dựng, chạy và quản lý các containers. Điều này tương tự như việc Git lưu trữ lịch sử của dự án dưới dạng một chuỗi các phiên bản (commits).

- Giao tiếp với Registry để pull hoặc push images. Điều này tương tự như việc Git push và pull thay đổi từ và đến remote repository.

- **Containers:**

- Là các instances độc lập được tạo từ images. Điều này tương tự như việc mỗi commit trong Git chứa thông tin về sự thay đổi trong mã nguồn cũng như thông tin về người tạo ra thay đổi và thời gian thực hiện.
- Có thể được start, stop, move, và delete. Điều này tương tự như việc Git cho phép người dùng tạo ra, hợp nhất, và quản lý các nhánh của dự án dễ dàng.

- **Images:**

- Là templates read-only dùng để tạo containers. Điều này tương tự như việc Git sử dụng một cấu trúc ba lớp gồm vùng làm việc, kho (local repository), và kho từ xa (remote repository).
- Có thể được build từ Dockerfile hoặc pulled từ Registry. Điều này tương tự như việc Git cho phép người dùng commit thay đổi vào local repository và sau đó push chúng lên remote repository.

- **Registry:**

- Nơi lưu trữ public hoặc private của images. Điều này tương tự như việc Git sử dụng các remote repository để lưu trữ và chia sẻ mã nguồn.
- Docker Hub là một ví dụ của Registry public. Điều này tương tự như việc GitHub là một ví dụ của remote repository trong Git.

3 Các câu lệnh cơ bản của Docker

Như đã nói Git là như một bộ công cụ linh hoạt và mạnh mẽ, giúp bạn kiểm soát hoàn toàn quy trình phát triển phần mềm. Từ việc theo dõi lịch sử mã nguồn đến việc quản lý các nhánh và hợp nhất các thay đổi, Git cung cấp mọi công cụ bạn cần để điều hành dự án của mình một cách hiệu quả. Các thao tác cơ bản sẽ được nêu lên ở bảng 1, ngoài ra có thể tham khảo thêm Git Tutorial[1] của trang học trực tuyến nổi tiến W3school

Bảng 1: Các câu lệnh Docker

	cú pháp	mô tả
1	<code>docker image pull <image_name></code>	Pull một image từ Docker Hub
2	<code>docker image ls</code>	Liệt kê các images hiện có
3	<code>docker image rm <image_name></code>	Xóa một image
4	<code>docker image build -t <image_name> .</code>	Build một image từ Dockerfile
5	<code>docker image history <image_name></code>	Xem lịch sử build của một image
6	<code>docker container run <image_name></code>	Tạo một container
7	<code>docker container start <container_name></code>	Start một container
8	<code>docker container stop <container_name></code>	Stop một container
9	<code>docker container rm <container_name></code>	Xóa một container
10	<code>docker container ps</code>	Liệt kê các container đang chạy
11	<code>docker container diff <container name></code>	Xem các thay đổi trên container

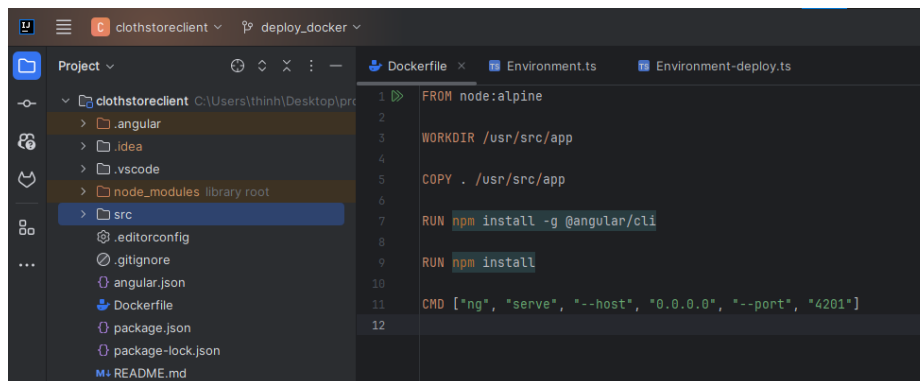
4 Thực nghiệm

Để hình dung các lý thuyết và câu lệnh đã được viết trong bài báo này. Phần thực nghiệm sẽ sử dụng Docker triển khai phần mềm. Đây là các phần cần chuẩn bị để thực nghiệm.

- **Docker Desktop**: Là một môi trường chứa đựng ứng dụng cùng với tất cả các thành phần cần thiết để chạy ứng dụng đó, bao gồm mã nguồn, thư viện và các tài nguyên khác. Containers là một cách tiêu biểu để đảm bảo tính nhất quán và di động của ứng dụng. Bạn có thể vào link này để xem cách install Docker Desktop cho Window:
<https://docs.docker.com/desktop/install/windows-install/>
- **WSL2**: Windows Subsystem for Linux 2 (WSL2) là một lớp tương thích cho phép chạy các hệ điều hành Linux trên Windows, giúp Docker Desktop hoạt động mượt mà hơn.
<https://learn.microsoft.com/en-us/windows/wsl/install-manual>
- **Ubuntu 22.04**: Một phiên bản phổ biến của hệ điều hành Linux, được sử dụng trong WSL2 để cung cấp môi trường Linux cho Docker. cài đặt Ubuntu 22.04 thực hiện bằng các lệnh sau :
wsl --list --online , wsl --install -d Ubuntu-22.04

4.1 Docker For Angular

Bước 1: Tạo Dockerfile

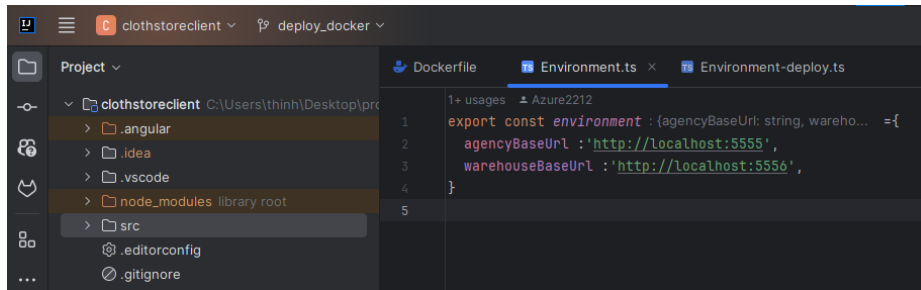


Hình 3: hình ảnh demo

- **FROM node:alpine**: Dòng này chỉ định image cơ sở mà Docker sẽ sử dụng để xây dựng image mới. Trong trường hợp này, image cơ sở được sử dụng là **node:alpine**, đây là một image nhẹ của Node.js được xây dựng trên Alpine Linux.
- **WORKDIR /usr/src/app**: Dòng này đặt thư mục làm việc mặc định cho các câu lệnh tiếp theo trong Dockerfile. Trong trường hợp này, thư mục **/usr/src/app** được đặt làm thư mục làm việc.
- **COPY . /usr/src/app**: Dòng này sao chép tất cả các file và thư mục từ thư mục hiện tại của host (thư mục mà Dockerfile đang được đặt) vào thư mục làm việc trong container, tức là **/usr/src/app**.
- **RUN npm install -g @angular/cli**: Dòng này chạy lệnh **npm install -g @angular/cli** trong container, cài đặt Angular CLI một cách toàn cục (-g) để có thể sử dụng trong toàn bộ hệ thống.

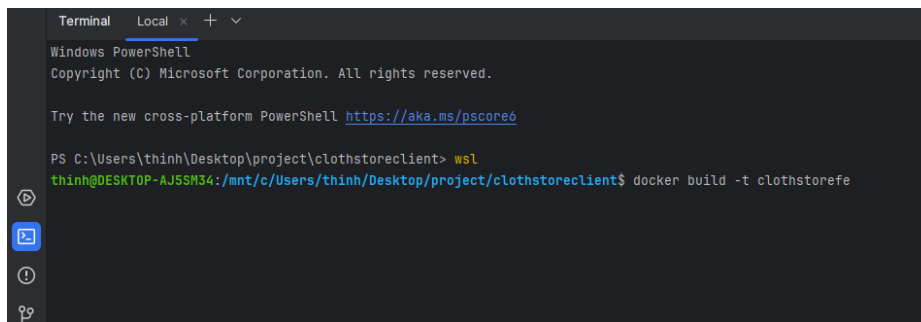
- **RUN npm install**: Dòng này chạy lệnh `npm install` trong container, cài đặt tất cả các dependencies của ứng dụng từ file `package.json` vào thư mục làm việc.
- **CMD ["ng", "serve", "-host", "0.0.0.0", "-port", "4201"]**: Dòng này chỉ định lệnh mặc định sẽ được chạy khi container được khởi chạy. Trong trường hợp này, nó sử dụng Angular CLI để khởi động dịch vụ phát triển Angular bằng lệnh `ng serve`, với các tùy chọn `-host 0.0.0.0` để cho phép truy cập từ bất kỳ địa chỉ IP và `-port 4201` để chỉ định cổng mà dịch vụ sẽ lắng nghe.

Bước 2: Kiểm tra Environment



Hình 4: hình ảnh demo

Bước 3: Viết lệnh để chạy Dockerfile



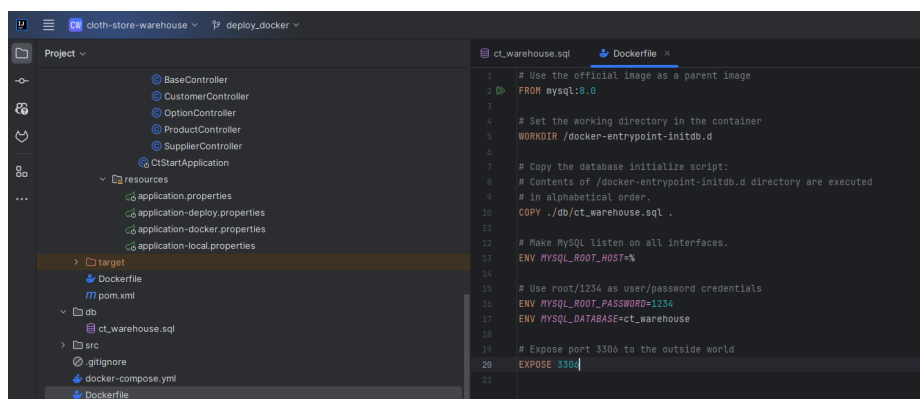
Hình 5: hình ảnh demo

- **wsl**: Là lệnh để chuyển sang Windows Subsystem for Linux (WSL) trên Windows. Trong trường hợp này, dường như người dùng đã chuyển sang WSL để sử dụng các lệnh của Linux. Điều này giúp thực hiện các lệnh Docker trên một môi trường Linux trên Windows.
- **docker build -t clothstorefe** : Lệnh này sẽ tạo ra một image mới từ Dockerfile được chỉ định (trong thư mục hiện tại) và gán tên cho image đó là `clothstorefe`. Dấu `-t` được sử dụng để chỉ định tên (tag) của image.

4.2 Docker For MySQL

Bước 1: Tạo Dockerfile

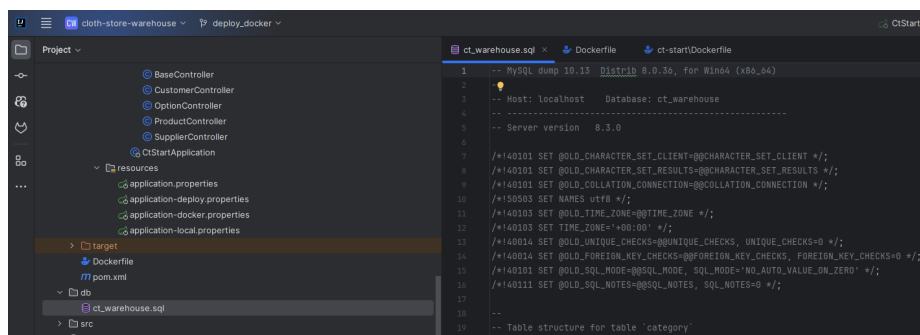
- **FROM mysql:8.0**: Dòng này sử dụng image chính thức của MySQL phiên bản 8.0 làm image cơ sở để xây dựng image mới.
- **WORKDIR /docker-entrypoint-initdb.d**: Dòng này đặt thư mục làm việc mặc định trong container là `/docker-entrypoint-initdb.d`. Thư mục này thường được sử dụng để chứa các tệp script SQL hoặc các tệp khởi tạo dữ liệu mà MySQL sẽ thực thi khi container khởi động.



Hình 6: hình ảnh demo

- **COPY ./db/ct_warehouse.sql :** Dòng này sao chép tệp `ct_warehouse.sql` từ thư mục `./db/` của host vào thư mục làm việc trong container. Điều này đảm bảo rằng tệp script SQL sẽ được thực thi khi container khởi động.
- **ENV MYSQL_ROOT_HOST=%:** Dòng này thiết lập biến môi trường `MYSQL_ROOT_HOST` với giá trị `%`, cho phép MySQL lắng nghe các kết nối từ tất cả các địa chỉ IP.
- **ENV MYSQL_ROOT_PASSWORD=1234:** Dòng này thiết lập biến môi trường `MYSQL_ROOT_PASSWORD` với giá trị `1234`, đây là mật khẩu cho tài khoản `root` của MySQL.
- **ENV MYSQL_DATABASE=ct_warehouse:** Dòng này thiết lập biến môi trường `MYSQL_DATABASE` với giá trị `ct_warehouse`, đây là tên của cơ sở dữ liệu sẽ được tạo khi container MySQL khởi động.
- **EXPOSE 3306:** Dòng này mô tả rằng container sẽ lắng nghe kết nối trên cổng `3306`, cổng mặc định của MySQL, và cho phép các ứng dụng khác có thể kết nối đến MySQL container thông qua cổng này. Tuy nhiên, điều này không tự động mở cổng này cho host, bạn cần phải cấu hình tường lửa hoặc mạng Docker để cho phép kết nối đến cổng này từ bên ngoài container.

Bước 2: Tạo tệp script SQL



Hình 7: hình ảnh demo

- Tạo một tệp `ct_warehouse.sql` chứa các lệnh SQL hoặc xuất từ cơ sở dữ liệu hiện có, đặt vào thư mục `./db/` trên host để Docker có thể sử dụng khi khởi động container.

Bước 3: Viết lệnh để chạy Dockerfile

```
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

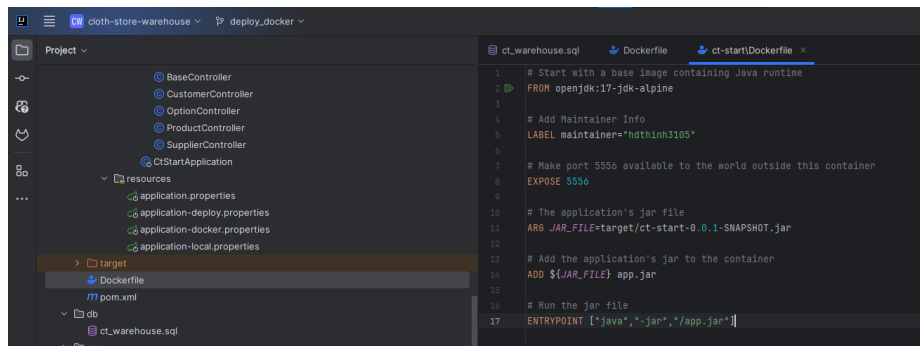
This message is shown once a day. To disable it please create the
/home/thinh/.hushlogin file.
thinh@DESKTOP-AJ5SH34:/mnt/c/Users/thinh/Desktop/project/cloth-store-warehouse$ docker build -t warehousedb
```

Hình 8: hình ảnh demo

- **docker build -t warehousedb** : Lệnh này sẽ tạo ra một image mới từ Dockerfile được chỉ định (trong thư mục hiện tại) và gán tên cho image đó là warehousedb. Dấu -t được sử dụng để chỉ định tên (tag) của image.

4.3 Docker For Spring Boot

Bước 1: Tạo Dockerfile



Hình 9: hình ảnh demo

- **FROM openjdk:17-jdk-alpine**: Dòng này sử dụng image chính thức của OpenJDK phiên bản 17 trên Alpine Linux làm image cơ sở để xây dựng image mới. Alpine Linux là một bản phân phối Linux nhẹ được sử dụng rộng rãi cho các container Docker vì kích thước nhỏ và hiệu suất cao.
- **LABEL maintainer="hdthinh3105"**: Dòng này thêm một nhãn (label) vào image, chỉ định thông tin về người duy trì image. Trong trường hợp này, nhãn được đặt là maintainer với giá trị là tên người duy trì image.
- **EXPOSE 5556**: Dòng này mô tả rằng container sẽ lắng nghe kết nối trên cổng 5556, cho phép các ứng dụng khác có thể kết nối đến container thông qua cổng này. Tuy nhiên, như đã nói trước đó, việc sử dụng EXPOSE không tự động mở cổng này cho host hoặc bên ngoài container.
- **ARG JAR_FILE=target/ct-start-0.0.1-SNAPSHOT.jar**: Dòng này định nghĩa một đối số build-time (ARG) có tên là JAR_FILE với giá trị mặc định là target/ct-start-0.0.1-SNAPSHOT.jar. Đối số này sẽ được sử dụng để chỉ định tệp jar của ứng dụng trong quá trình build.
- **ADD \$JAR_FILE app.jar**: Dòng này thêm tệp jar của ứng dụng vào container với tên app.jar. Tệp jar này được sao chép từ đường dẫn được chỉ định bởi biến JAR_FILE, có thể là một tệp jar được build từ mã nguồn của ứng dụng.
- **ENTRYPOINT ["java","-jar","/app.jar"]**: Dòng này định nghĩa lệnh chạy mặc định khi container được khởi động. Trong trường hợp này, nó sử dụng java -jar /app.jar để chạy ứng dụng Java đã được đóng gói trong tệp jar có tên là app.jar.

Bước 2: Đóng gói ứng dụng

- Mở Run Anything và gõ lệnh: `mvn clean package`

Bước 3: Viết lệnh để chạy Dockerfile



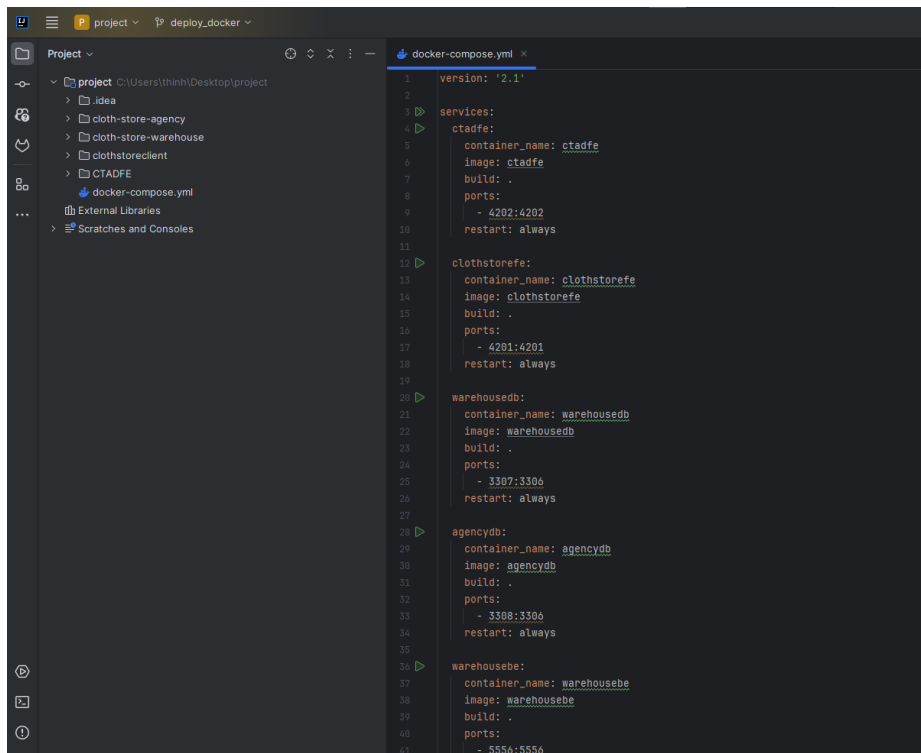
```
just raised the bar for easy, resilient and secure K8s cluster deployment.  
https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
This message is shown once a day. To disable it please create the  
/home/thinh/.hushlogin file.  
thinh@DESKTOP-AJSSM34:/mnt/c/Users/thinh/Desktop/project/cloth-store-warehouse$ docker build -t warehousebe .
```

Hình 10: hình ảnh demo

- **docker build -t warehousebe .**: Lệnh này sẽ tạo ra một image mới từ Dockerfile được chỉ định (trong thư mục hiện tại) và gán tên cho image đó là warehousebe. Dấu `-t` được sử dụng để chỉ định tên (tag) của image.

4.4 Docker Compose Configuration

Bước 1: Tạo Docker Compose File



```
1 version: '2.1'  
2  
3 services:  
4   ctadfe:  
5     container_name: ctadfe  
6     image: ctadfe  
7     build: .  
8     ports:  
9       - 4202:4202  
10    restart: always  
11  
12  clothstorefe:  
13    container_name: clothstorefe  
14    image: clothstorefe  
15    build: .  
16    ports:  
17      - 4201:4201  
18    restart: always  
19  
20  warehousedb:  
21    container_name: warehousedb  
22    image: warehousedb  
23    build: .  
24    ports:  
25      - 3307:3306  
26    restart: always  
27  
28  agencydb:  
29    container_name: agencydb  
30    image: agencydb  
31    build: .  
32    ports:  
33      - 3308:3306  
34    restart: always  
35  
36  warehousebe:  
37    container_name: warehousebe  
38    image: warehousebe  
39    build: .  
40    ports:  
41      - 5556:5556
```

Hình 11: hình ảnh demo

- **Version:**
 - * **version: '2.1'**: Chỉ định phiên bản của Docker Compose sẽ sử dụng. Trong trường hợp này, đây là phiên bản 2.1.
- **Services:**
 - * **ctadfe**:
 - + Dịch vụ này đặt tên là **ctadfe**, sẽ chạy một container sử dụng image **ctadfe**, được xây dựng từ thư mục hiện tại (do **build: .**).

- + Cổng 4202 của host sẽ được ánh xạ vào cổng 4202 của container, và container sẽ luôn được khởi động lại khi container dừng.
- * **clothstorefe:**
 - + Dịch vụ này đặt tên là **clothstorefe**, sẽ chạy một container sử dụng image **clothstorefe**, cấu hình tương tự như dịch vụ **ctadfe** nhưng ánh xạ cổng 4201 của host vào cổng 4201 của container.
- * **warehousedb và agencydb:**
 - + Đây là các dịch vụ cơ sở dữ liệu MySQL, mỗi dịch vụ được cấu hình để chạy một container MySQL, sử dụng image **warehousedb** và **agencydb**, tương ứng.
 - + Các cổng 3307 và 3308 của host sẽ được ánh xạ vào cổng 3306 của container MySQL tương ứng.
- * **warehousebe và agencybe:**
 - + Đây là các dịch vụ backend, được cấu hình tương tự như các dịch vụ frontend (**clothstorefe** và **ctadfe**).
 - + Mỗi dịch vụ sẽ chạy một container sử dụng image tương ứng và cổng được ánh xạ từ host vào container.
- Mỗi dịch vụ trong **docker-compose.yml** đều có các thuộc tính như **container_name**, **image**, **build**, **ports**, và **restart**.
- Các thuộc tính này cung cấp thông tin về cách cấu hình và chạy các containers tương ứng khi sử dụng Docker Compose.

Bước 2: Viết lệnh để chạy Docker Compose

```

Terminal  Local x Local (2) x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\thinh\Desktop\project> wsl
thinh@DESKTOP-AJ5SM34:/mnt/c/Users/thinh/Desktop/project$ docker-compose up

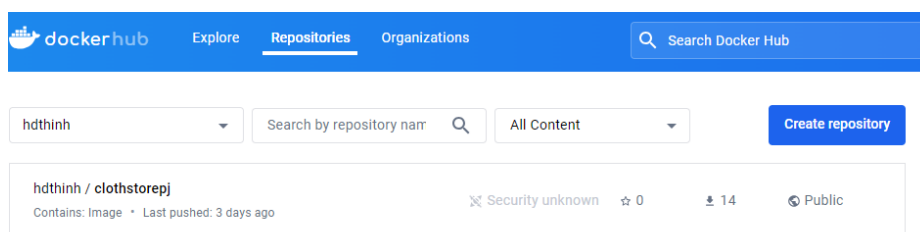
```

Hình 12: hình ảnh demo

- **docker-compose up**

4.5 Push Docker Images lên Docker Hub

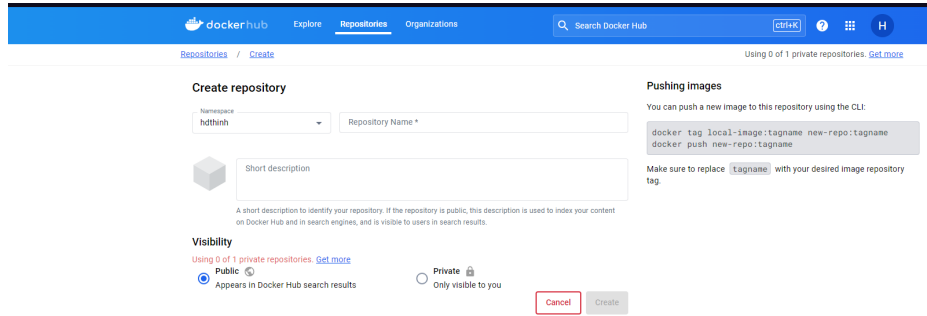
Bước 1: Tạo tài khoản trên Docker Hub



Hình 13: hình ảnh demo

- Bạn có thể tạo tài khoản thông qua link này: <https://hub.docker.com/signup>

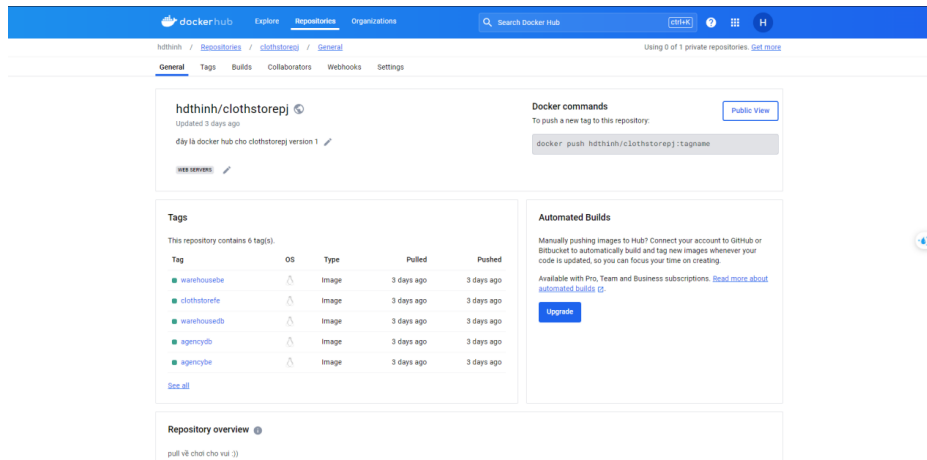
Bước 2: Tạo Repository



Hình 14: hình ảnh demo

- Sau khi đăng ký và đăng nhập, bạn có thể tạo một repository mới trên Docker Hub.

Bước 3: Push Images lên Repository

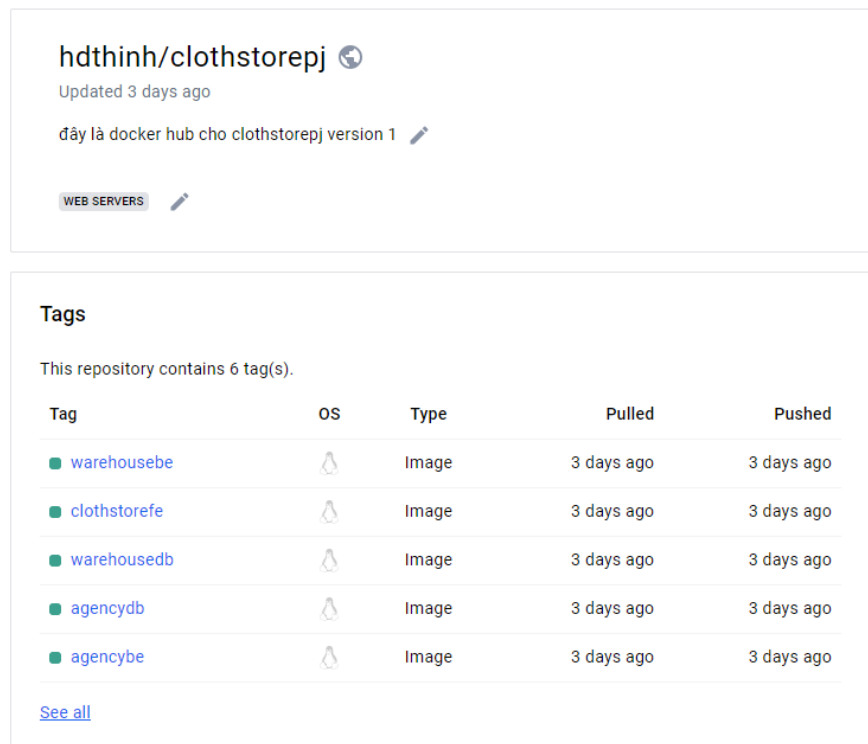


Hình 15: hình ảnh demo

- Sau khi tạo xong repository, bạn có thể push các image có trong Docker Desktop lên repository bạn mới tạo bằng các lệnh trong terminal:
 - * `docker login`
 - * `docker tag local-image:tagname new-repo:tagname`
 - + Ví dụ: `docker tag warehousedb hdthinh/clothstorepj:warehousedb`
 - + Chú ý: `hdthinh/clothstorepj` phải trùng với tên repository.
 - * `docker push new-repo:tagname`
 - + Ví dụ: `docker push hdthinh/clothstorepj:warehousedb`

Bước 4: Kiểm tra Repository

- Vào repository trên Docker Hub và kiểm tra xem các image đã được đẩy lên hay chưa.
- Bạn có thể vào phần tag để xem các lệnh pull image.



Hình 16: hình ảnh demo

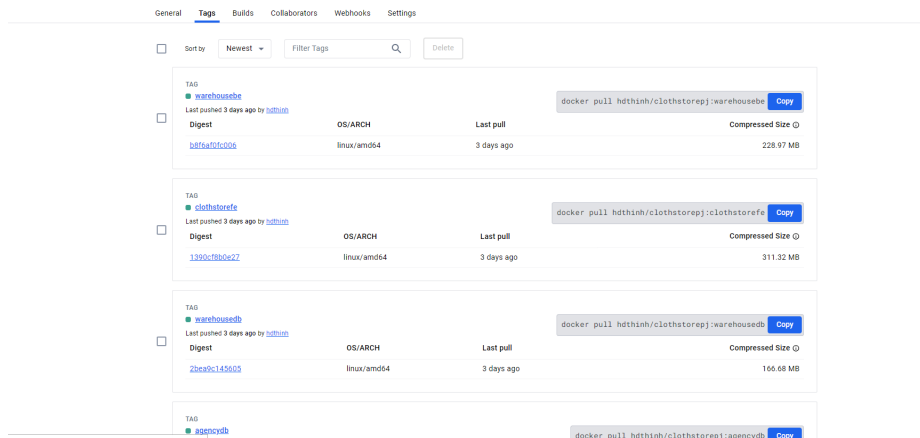
Bước 5: Sửa Docker Compose File

- Nếu bạn muốn chạy các image này từ repository, chỉ cần sửa lại tên image trong `docker-compose.yml` để sử dụng image từ Docker Hub.

4.6 Docker For ASP.NET

Bước 1: Tạo Dockerfile

- **FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base:**
 - * Sử dụng image chính thức của .NET 8.0 ASP.NET runtime làm image cơ bản.
- **USER app:**
 - * Thiết lập người dùng là `app` để chạy ứng dụng, giúp tăng cường bảo mật.
- **WORKDIR /app:**
 - * Đặt thư mục làm việc bên trong container là `/app`.
- **EXPOSE 8080:**
 - * Mở cổng 8080 để cho phép giao tiếp với container.
- **FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build:**
 - * Sử dụng image .NET SDK 8.0 để xây dựng ứng dụng.
- **ARG BUILD_CONFIGURATION=Release:**
 - * Định nghĩa một biến cấu hình xây dựng có tên là `BUILD_CONFIGURATION` với giá trị mặc định là `Release`.
- **WORKDIR /src:**
 - * Đặt thư mục làm việc bên trong container là `/src` cho quá trình xây dựng.



Hình 17: hình ảnh demo

```

1 #See https://aka.ms/customizecontainer to learn how to customize your debug container and how Visual Studio uses this Dockerfile to build your
2
3 FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
4 USER app
5 WORKDIR /app
6 EXPOSE 8080
7
8 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
9 ARG BUILD_CONFIGURATION=Release
10 WORKDIR /src
11 COPY ["BlogBE/BlogBE.csproj", "BlogBE/"]
12 RUN dotnet restore "./BlogBE/BlogBE.csproj"
13 COPY . .
14 WORKDIR "/src/BlogBE"
15 RUN dotnet build "./BlogBE.csproj" -c $BUILD_CONFIGURATION -o /app/build
16
17 # Generate a self-signed certificate and export it to a .pfx file
18 RUN dotnet dev-certs https -ep /https/aspnetapp.pfx -p 123456
19 RUN chmod 600 /https/aspnetapp.pfx
20 RUN chown app:app /https/aspnetapp.pfx
21
22 FROM build AS publish
23 ARG BUILD_CONFIGURATION=Release
24 RUN dotnet publish "BlogBE.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false
25
26 FROM base AS final
27 WORKDIR /app
28 COPY --from=publish /app/publish
29 # Copy the pfx file into the Docker image
30 COPY --from=build /https/aspnetapp.pfx /https/
31
32 # Configure the application to use the certificate
33 ENV ASPNETCORE_Microsoft__Certificates__Default_Password="123456"
34 ENV ASPNETCORE_Microsoft__Certificates__Default_Path="/https/aspnetapp.pfx"
35
36 ENV ASPNETCORE_URLS=https://+:8080
37
38 ENTRYPOINT ["dotnet", "BlogBE.dll"]
39

```

Hình 18: hình ảnh demo

- **COPY ["BlogBE/BlogBE.csproj", "BlogBE/"]:**
 - * Sao chép file dự án BlogBE.csproj vào thư mục BlogBE trong container.
- **RUN dotnet restore "./BlogBE/BlogBE.csproj":**
 - * Khôi phục các gói phụ thuộc của dự án được xác định trong file BlogBE.csproj.
- **COPY . .:**
 - * Sao chép tất cả các file từ thư mục hiện tại trên máy chủ vào thư mục hiện tại trong container.
- **WORKDIR "/src/BlogBE":**
 - * Thay đổi thư mục làm việc thành /src/BlogBE bên trong container.
- **RUN dotnet build "./BlogBE.csproj \$BUILD_CONFIGURATION -o /app/build:**
 - * Xây dựng dự án bằng cấu hình xây dựng được chỉ định (mặc định là Release) và xuất kết quả xây dựng vào thư mục /app/build.
- **RUN dotnet dev-certs https -ep /https/aspnetapp.pfx -p 123456:**
 - * Tạo chứng chỉ HTTPS tự ký và xuất nó vào file /https/aspnetapp.pfx với mật khẩu 123456.
- **RUN chmod 600 /https/aspnetapp.pfx:**

- * Đặt quyền truy cập cho file chứng chỉ để chỉ người dùng sở hữu mới có thể đọc và ghi.
- **RUN chown app:app /https/aspnetapp.pfx:**
 - * Thay đổi quyền sở hữu file chứng chỉ cho người dùng **app**.
- **FROM build AS publish:**
 - * Định nghĩa một stage mới tên là **publish** sử dụng stage **build** làm cơ sở.
- **ARG BUILD_CONFIGURATION=Release:**
 - * Định nghĩa lại biến **BUILD_CONFIGURATION** cho stage **publish**.
- **RUN dotnet publish "/BlogBE.csproj \$BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false:**
 - * Xuất bản dự án bằng cấu hình xây dựng được chỉ định và xuất kết quả xuất bản vào thư mục **/app/publish** mà không sử dụng **app host**.
- **FROM base AS final:**
 - * Định nghĩa stage cuối cùng tên là **final** sử dụng stage **base** làm cơ sở.
- **WORKDIR /app:**
 - * Đặt thư mục làm việc bên trong container là **/app**.
- **COPY --from=publish /app/publish .:**
 - * Sao chép các file ứng dụng đã xuất bản từ stage **publish** vào thư mục hiện tại trong stage **final**.
- **COPY --from=build /https/aspnetapp.pfx /https/:**
 - * Sao chép chứng chỉ HTTPS từ stage **build** vào thư mục **/https/** trong stage **final**.
- **ENV ASPNETCORE_Kestrel__Certificates__Default__Password="123456":**
 - * Thiết lập biến môi trường để chỉ định mật khẩu cho chứng chỉ HTTPS.
- **ENV ASPNETCORE_Kestrel__Certificates__Default__Path="/https/aspnetapp.pfx"**
 - * Thiết lập biến môi trường để chỉ định đường dẫn đến file chứng chỉ HTTPS.
- **ENV ASPNETCORE_URLS=https://+:8080:**
 - * Thiết lập biến môi trường để cấu hình ứng dụng lắng nghe trên HTTPS tại cổng 8080.
- **ENTRYPOINT ["dotnet", "BlogBE.dll"]:**
 - * Thiết lập entry point cho container để chạy ứng dụng .NET, cụ thể là file **BlogBE.dll**.

Bước 2: Viết lệnh để chạy Dockerfile và Image

```

thinh@DESKTOP-N48D41B: /mnt/d/Project 2023-2024/Blog_ASP.net_SQLServer
Welcome to Ubuntu 20.04 LTS (GNU/Linux 5.15.146.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Mon May 27 17:04:26 +07 2024

System load:  0.0          Processes:    16
Usage of /:   0.4% of 250.92GB   Users logged in:  0
Memory usage: 9%            IPv4 address for eth0: 172.22.34.39
Swap usage:   0%

0 updates can be installed immediately.
0 of these updates are security updates.

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

This message is shown once a day. To disable it please create the
thinh@DESKTOP-N48D41B:/mnt/d/Project 2023-2024/Blog_ASP.net_SQLServer$ docker build --no-cache -t blogbe2 -f BlogBE/Dockerfile1 .

```

Hình 19: hình ảnh demo

```
thinh@DESKTOP-H48D41B: /mnt/d/Project 2023-2024/Blog_ASP.net_SQLServer$
Welcome to Ubuntu 20.04 LTS (GNU/Linux 5.15.146.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Mon May 27 17:04:26 +07 2024

System load:  0.0          Processes:      16
Usage of /:   0.4% of 250.92GB   Users logged in: 0
Memory usage: 9%             IPv4 address for eth0: 172.22.34.39
Swap usage:   0%

0 updates can be installed immediately.
0 of these updates are security updates.

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

This message is shown once once a day. To disable it please create the
thinh@DESKTOP-H48D41B:/mnt/d/Project 2023-2024/Blog_ASP.net_SQLServer$ docker run -d -p 8080:8080 -e ASPNETCORE_ENVIRONMENT=Development blogbe2
```

Hình 20: hình ảnh demo

- **docker build --no-cache -t blogbe -f BlogBE/Dockerfile .:**
 - * Lệnh này sẽ tạo ra một image mới từ Dockerfile được chỉ định (trong thư mục hiện tại) và gán tên cho image đó là **blogbe**. Dấu **-t** được sử dụng để chỉ định tên (tag) của image.
- **docker run -d -p 8080:8080 -e ASPNETCORE_ENVIRONMENT=Development blogbe:**
 - * Lệnh này sẽ chạy container từ image **blogbe**, mở cổng 8080 trên host và ánh xạ vào cổng 8080 trong container, đồng thời thiết lập biến môi trường **ASPNETCORE_ENVIRONMENT** là **Development**.

4.7 Deploy Docker Images lên Server

4.7.1 Deploy Docker Image warehousedb và agencydb lên ECS của AWS (ví dụ với warehousedb, agencydb làm tương tự)

Bước 1: Build image warehousedb

```
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

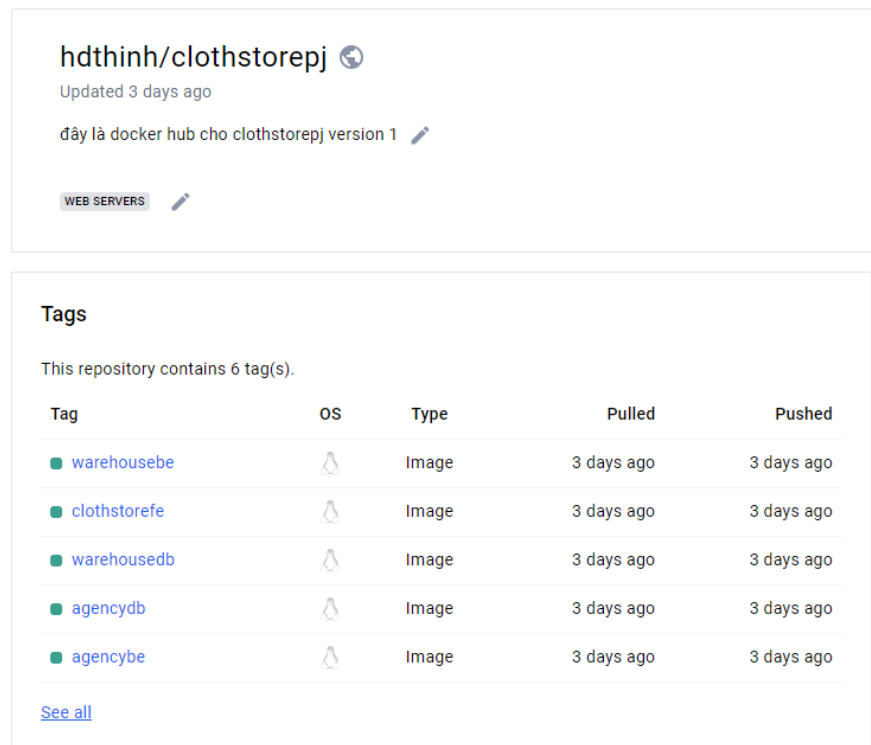
This message is shown once a day. To disable it please create the
/home/thinh/.hushlogin file.

thinh@DESKTOP-AJ5SM34:/mnt/c/Users/thinh/Desktop/project/cloth-store-warehouse$ docker build -t warehousedb
```

Hình 21: hình ảnh demo

- Build Docker image cho **warehousedb** bằng lệnh:
docker build -t warehousedb .

Bước 2: Push image lên repository đã tạo trong Docker Hub

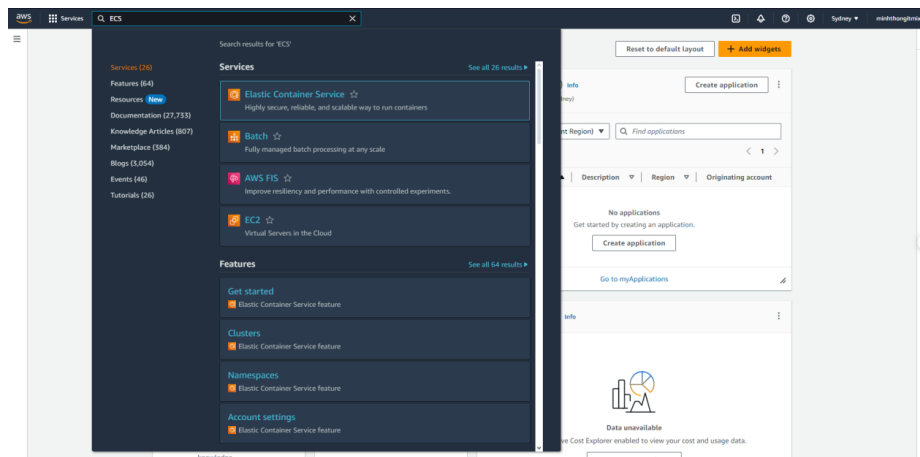


Hình 22: hình ảnh demo

- Tag và push image lên Docker Hub:

```
docker tag warehousedb hdthinh/clothstorepj:warehousedb
docker push hdthinh/clothstorepj:warehousedb
```

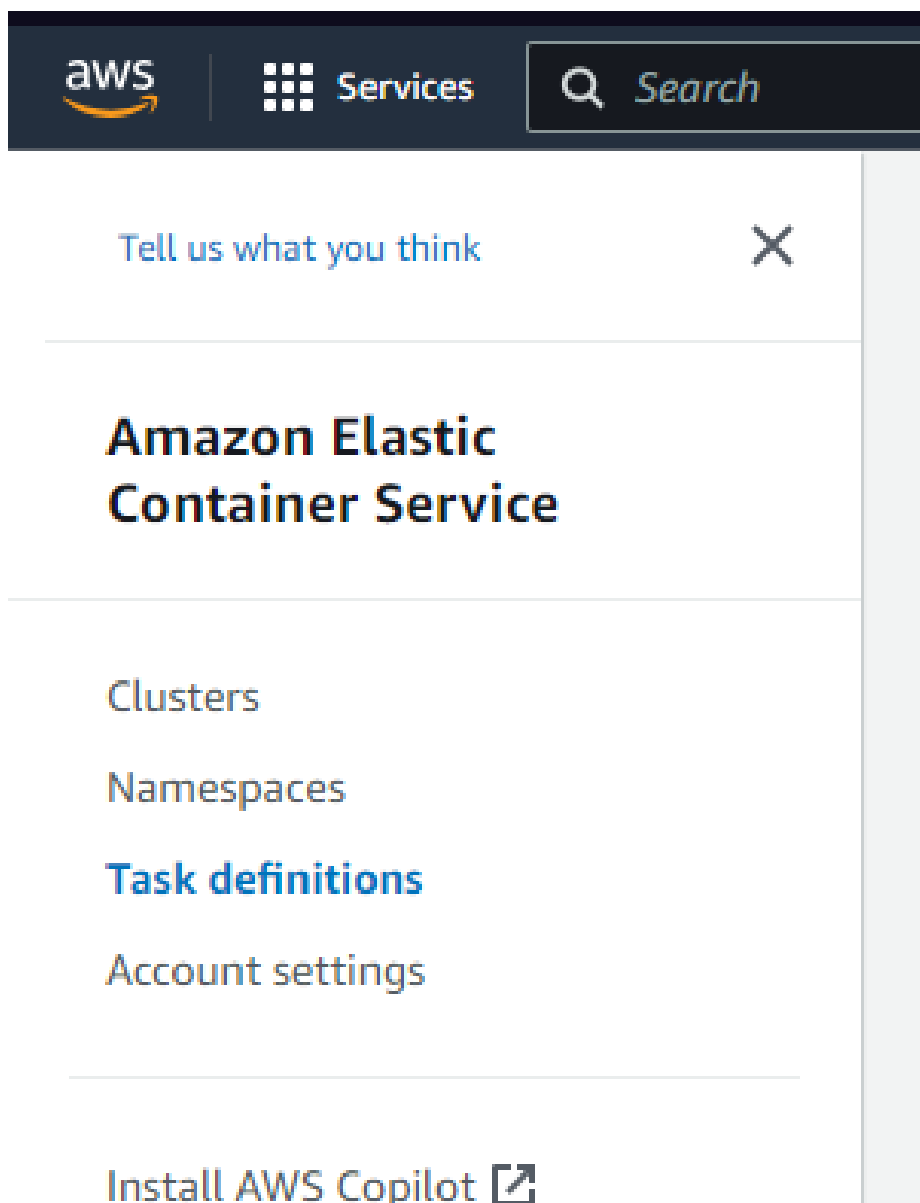
Bước 3: Tìm ECS trong tìm kiếm của AWS



Hình 23: hình ảnh demo

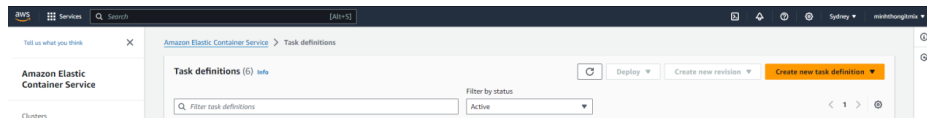
- Mở AWS Management Console và tìm kiếm ECS (Elastic Container Service).

Bước 4: Tạo Task definition cho warehousedb



Hình 24: hình ảnh demo

- Chọn **Task Definitions** và nhấn **Create new Task Definition**.
- Chọn **EC2** hoặc **Fargate** và nhấn **Next step**.
- Cấu hình container cho **warehousedb**, bao gồm tên, image từ Docker Hub, cổng, và các thiết lập khác.
- Nhấn **Create** ở cuối trang.



Hình 25: hình ảnh demo

Task definition configuration

Task definition family [Info](#)
Specify a unique task definition family name.
warehousedb
Up to 255 letters (uppercase and lowercase), numbers, hyphens, and underscores are allowed.

Revision
Source revision
3

▼ Infrastructure requirements
Specify the infrastructure requirements for the task definition.

Launch type [Info](#)
Selection of the launch type will change task definition parameters.

☒ AWS Fargate
Serverless compute for containers.

☐ Amazon EC2 instances
Self-managed infrastructure using Amazon EC2 instances.

OS, Architecture, Network mode
Network mode is used for tasks and is dependent on the compute type selected.

Operating system/Architecture [Info](#)
Linux/X86_64

Network mode [Info](#)
awsvpc

Task size [Info](#)
Specify the amount of CPU and memory to reserve for your task.

CPU
1 vCPU

Memory
3 GB

▼ Task roles - conditional

Hình 26: hình ảnh demo

▼ Task roles - conditional

Task role [Info](#)
A task IAM role allows containers in the task to make API requests to AWS services. You can create a task IAM role from the [IAM console](#).

None

Task execution role [Info](#)
A task execution IAM role is used by the container agent to make AWS API requests on your behalf. If you don't already have a task execution IAM role created, we can create one for you.

ecsTaskExecutionRole

▼ Task placement - optional

Task placement constraints are not supported for AWS Fargate launch type.

▼ Container - 1 [Info](#) Essential container Remove

Container details
Specify a name, container image, and whether the container should be marked as essential. Each task definition must have at least one essential container.

Name
warehousedb

Image URI
docker.io/hdthinh/clothstorepj:warehousedb

Essential container
Yes

Private registry [Info](#)
Store credentials in Secrets Manager, and then use the credentials to reference images in private registries.

☐ Private registry authentication

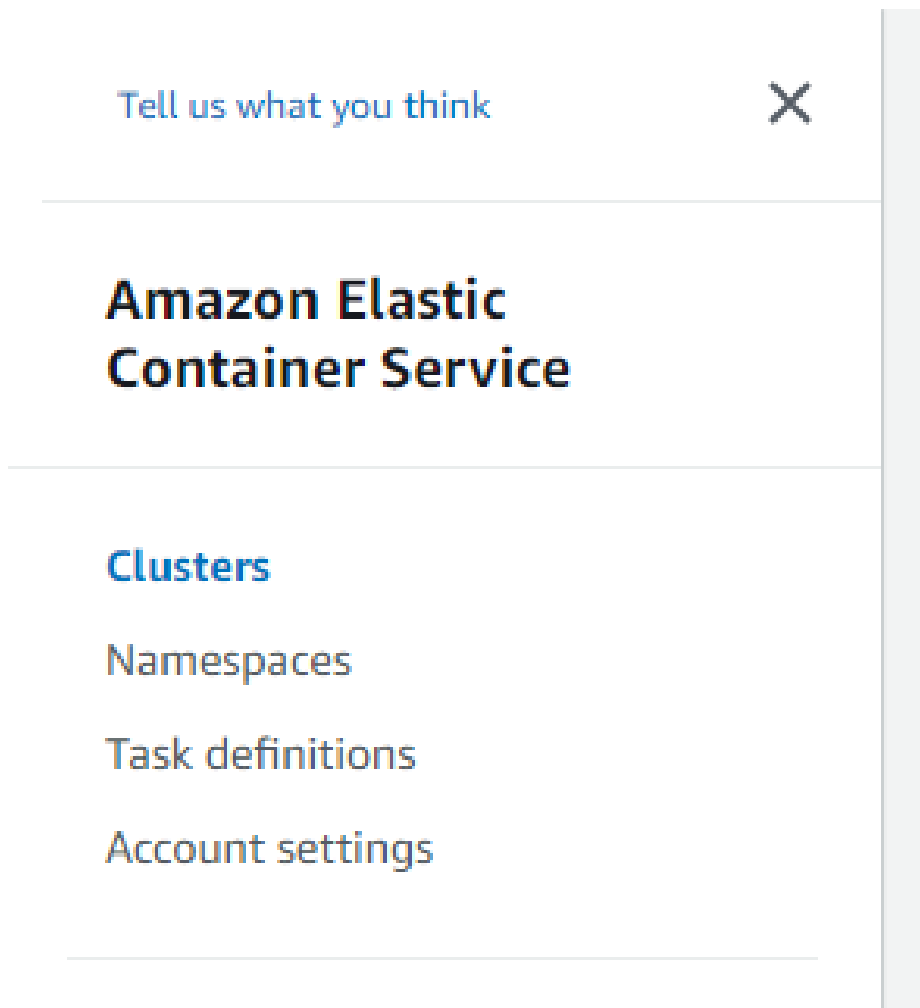
Port mappings [Info](#)
Add port mappings to allow the container to access ports on the host to send or receive traffic. For port name, a default will be assigned if left blank.

Container port	Protocol	Port name	App protocol	
3307	TCP	warehousedb-3307-tcp	HTTP	Remove

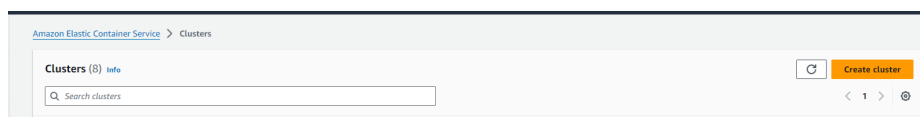
Add port mapping

Hình 27: hình ảnh demo

Bước 5: Tạo cluster



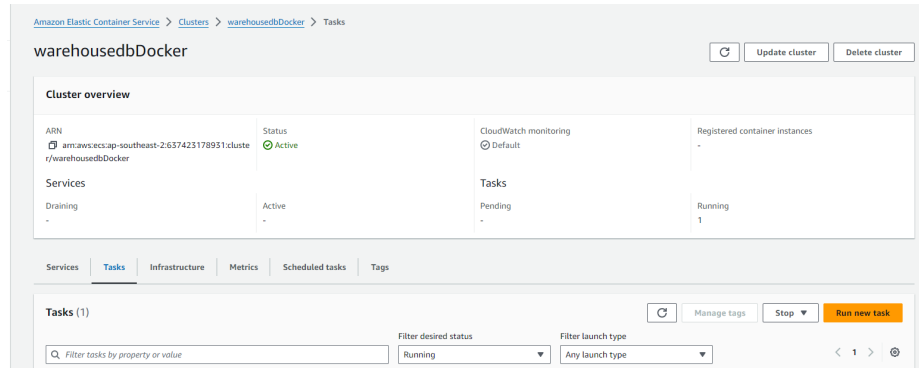
Hình 28: hình ảnh demo



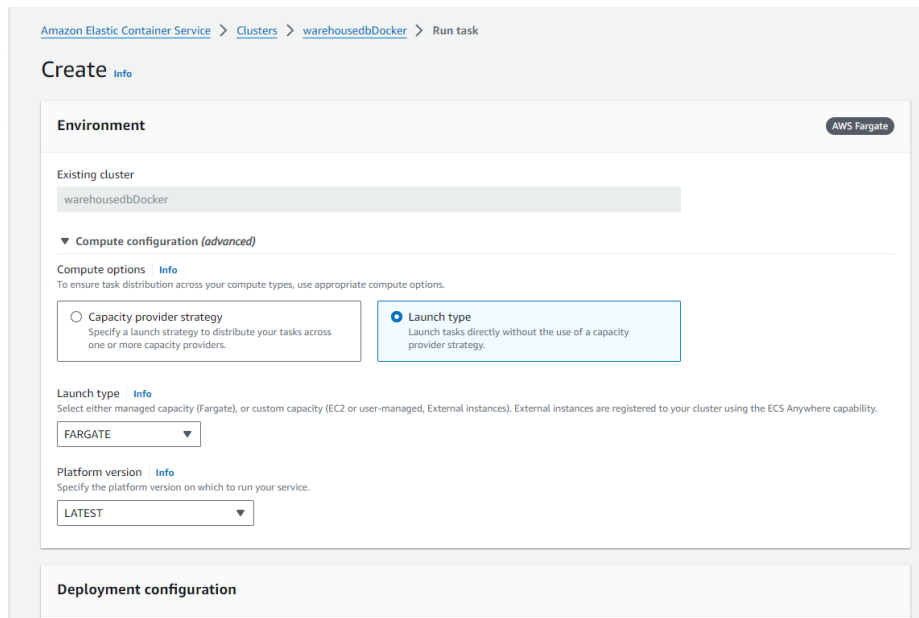
Hình 29: hình ảnh demo

- Chọn **Clusters** và nhấn **Create Cluster**.
- Chọn **Networking Only** hoặc **EC2 Linux + Networking**.
- Đặt tên cho cluster và nhấn **Create**.

Bước 6: Tạo task trong cluster mới vừa tạo



Hình 30: hình ảnh demo



Hình 31: hình ảnh demo

- Quay lại cluster mới tạo và chọn **Tasks** sau đó nhấn **Run New Task**.
- Chọn Task Definition đã tạo, cấu hình các tùy chọn cần thiết, và nhấn **Run Task**.

Deployment configuration

Application type [Info](#)

Specify what type of application you want to run.

☐ Service

Launch a group of tasks handling a long-running computing work that can be stopped and restarted. For example, a web application.

☒ Task

Launch a standalone task that runs and terminates. For example, a batch job.

Task definition

Select an existing task definition. To create a new task definition, go to [Task definitions](#).

☐ Specify the revision manually

Manually input the revision instead of choosing from the 100 most recent revisions for the selected task definition family.

Family

Select a task definition family

Revision

Desired tasks

Specify the number of tasks to launch.

1

Task group

All tasks with the same task group name are considered as a set when performing spread placement.

▼ Networking

VPC [Info](#)

Choose the Virtual Private Cloud to use.

vpc-03dd866f5c0e879b6

default

Subnets

Choose the subnets within the VPC that the task scheduler should consider for placement.

Hình 32: hình ảnh demo

▼ Networking

VPC [Info](#)

Choose the Virtual Private Cloud to use.

vpc-03dd866f5c0e879b6

default

Subnets

Choose the subnets within the VPC that the task scheduler should consider for placement.

Choose subnets

Clear current selection

subnet-0151f4f74deb5c82f
ap-southeast-2a 172.31.0.0/20

subnet-0a48cbaeda8e0b4e
ap-southeast-2b 172.31.32.0/20

subnet-0b3167960ae36dcc2
ap-southeast-2c 172.31.16.0/20

Security group [Info](#)

Choose an existing security group or create a new security group.

☒ Use an existing security group

☐ Create a new security group

Security group name

Choose an existing security group.

Choose security groups

sg-0b966a1802ba4b08a

default

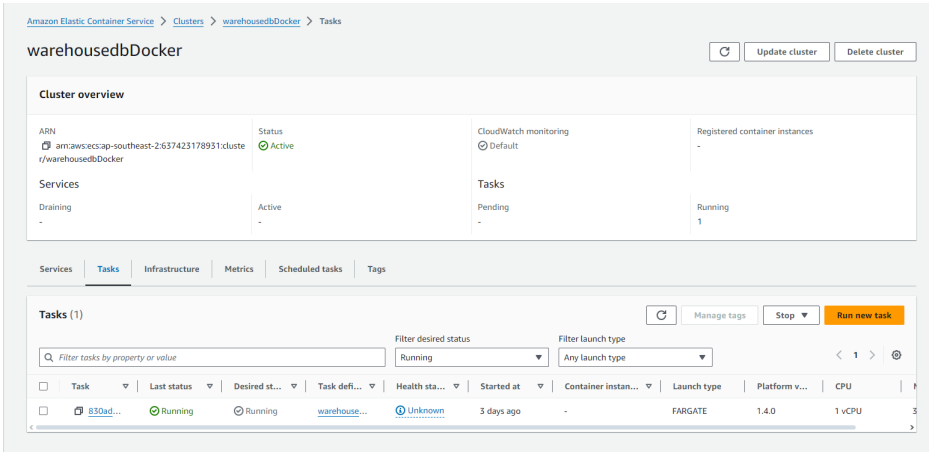
Public IP [Info](#)

Choose whether to auto-assign a public IP to the task's elastic network interface (ENI).

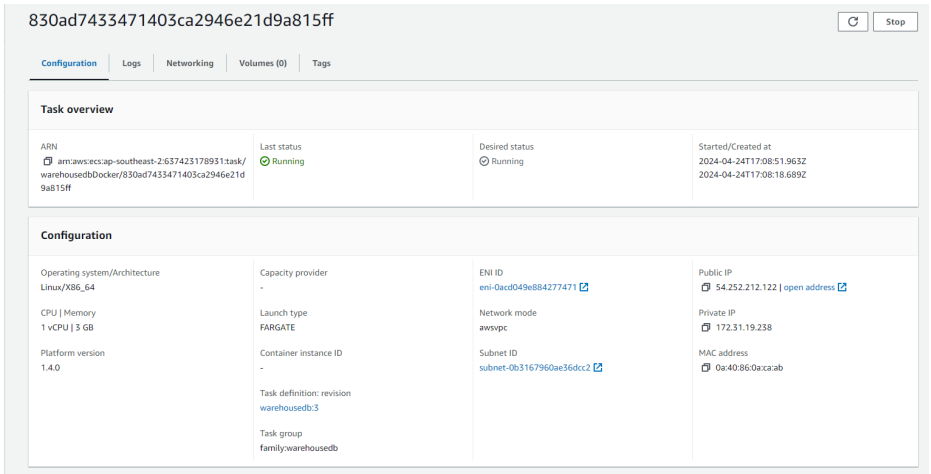
☒ Turned on

Hình 33: hình ảnh demo

Bước 7: Chờ task run xong và mở MySQL Workbench để check connection

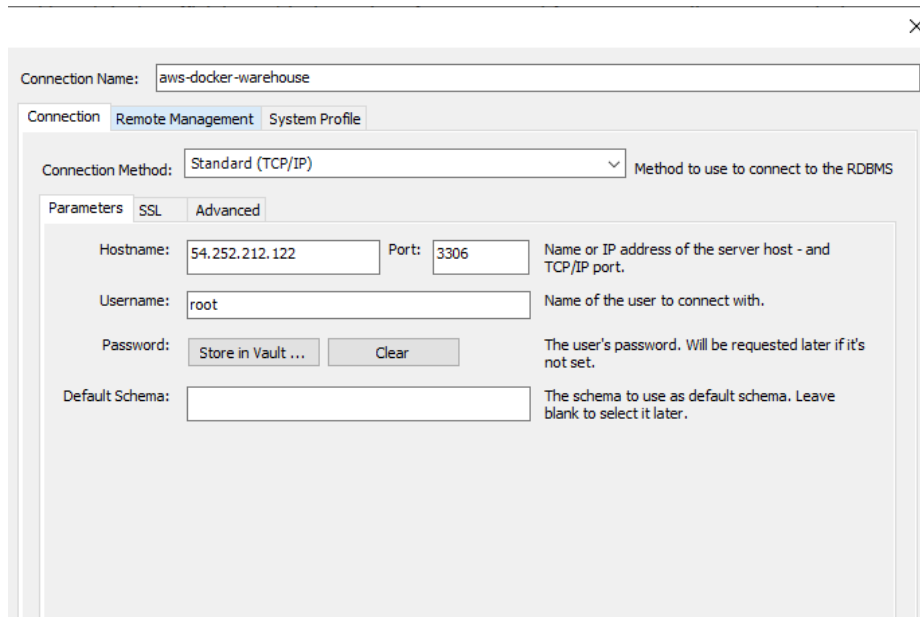


Hình 34: hình ảnh demo



Hình 35: hình ảnh demo

- Sau khi task chạy xong, lấy địa chỉ IP của container và kết nối bằng MySQL Workbench để kiểm tra.



The screenshot shows the 'Connection Name' dialog box in MySQL Workbench. The 'Connection Name' field is set to 'aws-docker-warehouse'. The 'Connection Method' is set to 'Standard (TCP/IP)'. The 'Parameters' tab is selected, showing the following fields: 'Hostname' (54.252.212.122), 'Port' (3306), 'Username' (root), 'Password' (with 'Store in Vault ...' and 'Clear' buttons), and 'Default Schema' (empty). The 'Remote Management' and 'System Profile' tabs are also visible.

Connection Name: aws-docker-warehouse

Connection Method: Standard (TCP/IP) Method to use to connect to the RDBMS

Parameters SSL Advanced

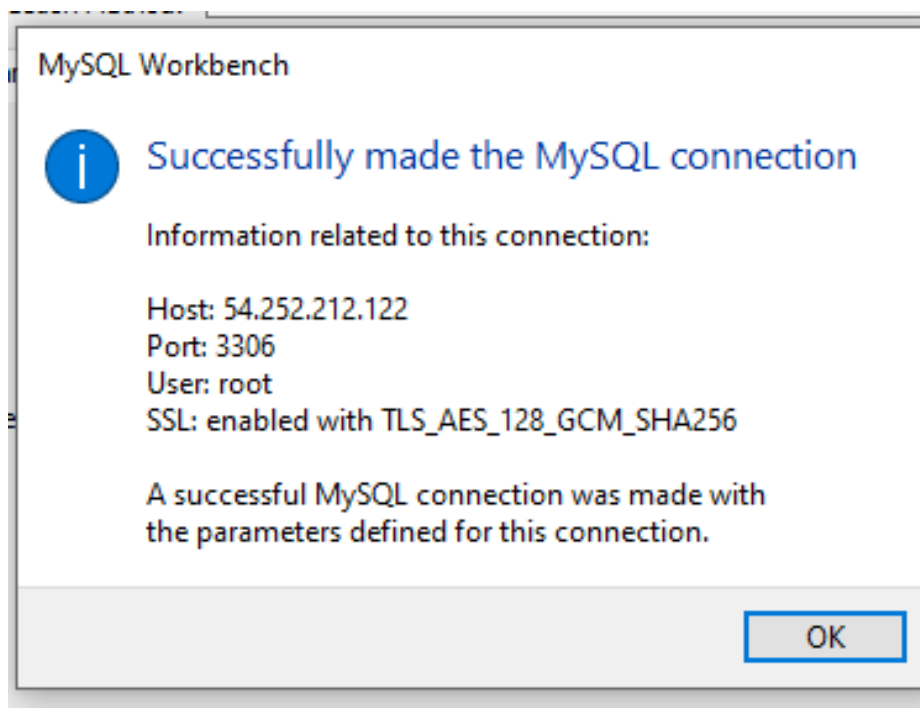
Hostname: 54.252.212.122 Port: 3306 Name or IP address of the server host - and TCP/IP port.

Username: root Name of the user to connect with.

Password: Store in Vault ... Clear The user's password. Will be requested later if it's not set.

Default Schema: The schema to use as default schema. Leave blank to select it later.

Hình 36: hình ảnh demo

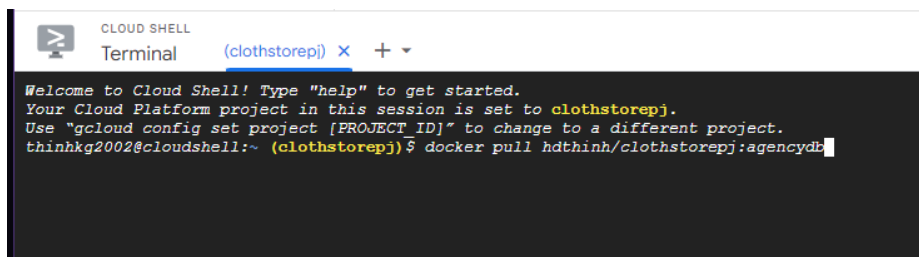


Hình 37: hình ảnh demo

4.7.2 Deploy Docker Image warehousedb và agencydb lên Kubernetes Engine GCloud (ví dụ với agencydb, warehousedb làm tương tự)

Bước 1: Thực hiện lại bước 1,2 của mục 7.a)

Bước 2: Pull image về GCloud



```
CLOUD SHELL
Terminal (clothstorepj) X + v

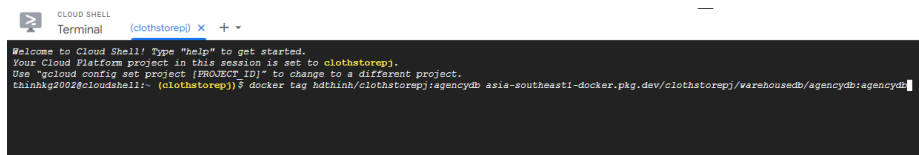
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to clothstorepj.
Use "gcloud config set project [PROJECT_ID]" to change to a different project.
thinhkg2002@cloudshell:~ (clothstorepj) $ docker pull hdthinh/clothstorepj:agencydb
```

Hình 38: hình ảnh demo

- Đăng nhập vào Google Cloud Shell và pull image từ Docker Hub:

```
docker pull hdthinh/clothstorepj:agencydb
```

Bước 3: Thay tag image



```
CLOUD SHELL
Terminal (clothstorepj) X + v

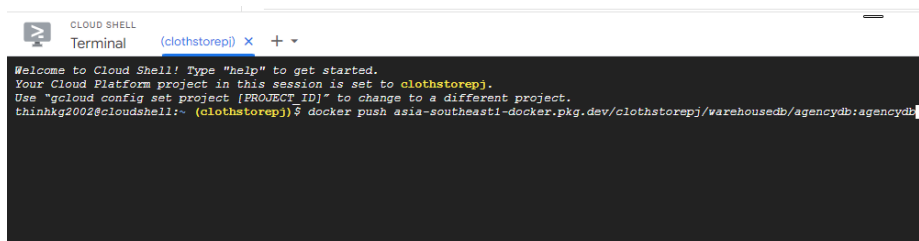
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to clothstorepj.
Use "gcloud config set project [PROJECT_ID]" to change to a different project.
thinhkg2002@cloudshell:~ (clothstorepj) $ docker tag hdthinh/clothstorepj:agencydb asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/agencydb:agencydb
```

Hình 39: hình ảnh demo

- Thay đổi tag của image:

```
docker tag hdthinh/clothstorepj:agencydb gcr.io/YOUR_PROJECT_ID/agencydb:latest
```

Bước 4: Push lên Artifact Registry



```
CLOUD SHELL
Terminal (clothstorepj) X + v

Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to clothstorepj.
Use "gcloud config set project [PROJECT_ID]" to change to a different project.
thinhkg2002@cloudshell:~ (clothstorepj) $ docker push asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/agencydb:agencydb
```

Hình 40: hình ảnh demo

- Push image lên Artifact Registry:

```
docker push gcr.io/YOUR_PROJECT_ID/agencydb:latest
```

Bước 5: Tạo cluster mới trong Kubernetes Engine

- Mở Google Kubernetes Engine và nhấn Create Cluster.
- Đặt tên cho cluster và cấu hình các tùy chọn cần thiết, sau đó nhấn Create.

Bước 6: Deploy trong cluster mới tạo

Google Cloud

ClothstorePJ

ar

←

Create an Autopilot cluster

Cluster basics

Set up basics for your cluster

Fleet registration

Manage multiple clusters together

Networking

Define applications communication in the cluster

Advanced settings

Review additional options

Review and create

Review all settings and create your cluster

Cluster basics

Create an Autopilot cluster by specifying a name and region. After the cluster is created, you can deploy your workload through Kubernetes and we'll take care of the rest, including:

- ✓ **Nodes:** Automated node provisioning, scaling, and maintenance
- ✓ **Networking:** VPC-native traffic routing for public or private clusters
- ✓ **Security:** Shielded GKE Nodes and Workload Identity
- ✓ **Telemetry:** Cloud Operations logging and monitoring

Name

agencydb

Cluster names must start with a lowercase letter followed by up to 39 lowercase letters, numbers, or hyphens. They can't end with a hyphen. You cannot change the cluster's name once it's created.

Region

asia-southeast1

The regional location in which your cluster's control plane and nodes are located. You cannot change the cluster's region once it's created.

NEXT: FLEET REGISTRATION

⌛ RESET SETTINGS

CREATE

CANCEL

Equ

Hình 41: hình ảnh demo

Google Cloud

ClothstorePJ

ar

Kubernetes Engine

← Clusters

EDIT

DELETE

DEPLOY

CONNECT

DUPLICATE

Learn about Enterprise

All Fleets

mysql2

DETAILS

STORAGE

OBSERVABILITY

LOGS

APP ERRORS (13)

Hình 42: Hình ảnh demo 1

Kubernetes Engine

[Learn about Enterprise](#)

All Fleets

Resource Management

- Overview
- Clusters
- Workloads

Teams NEW

Applications

Secrets & ConfigMaps

Storage

Object Browser

Backup for GKE

Posture Management

- Security
- Compliance PREVIEW
- Policy

Marketplace

Release Notes

Create a deployment

1 Container

New container

☒ Existing container image

☐ New container image

Image path *

nginx:latest

SELECT

Enter your image path, or choose from Google Container Registry. You can also enter the name of any public image from Docker Hub, such as nginx:latest. [Explore Docker Hub](#)

Environment variables

+ ADD ENVIRONMENT VARIABLE

Initial command

Overrides the default entrypoint of the container image.

DONE

ADD A CONTAINER

CONTINUE

2 Configuration

3 Expose (optional)

Hình 43: Hình ảnh demo 2

25

Select container image

ARTIFACT REGISTRY

CONTAINER REGISTRY

Project: clothstorepj

CHANGE

▼ asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb

▶ agencybe

▼ agencydb

2ff11103de

agencydb

5 hours ago

▶ clothstorefe

▶ ctadfe

▶ warehousebe

▶ warehousedb

SELECT

CANCEL

Hình 44: Hình ảnh demo 3

Container

New container

☒ Existing container image

☐ New container image

Image path *

asia-southeast1-docker.pkg.dev/clothstorepj/warehouse

SELECT

Enter your image path, or choose from Google Container Registry. You can also enter the name of any public image from Docker Hub, such as nginx:latest. [Explore Docker Hub](#)

Environment variables

+ ADD ENVIRONMENT VARIABLE

Initial command

Overrides the default entrypoint of the container image.

DONE

ADD A CONTAINER

CONTINUE

Hình 45: Hình ảnh demo 4

27

2 Configuration

A deployment is a configuration which defines how Kubernetes deploys, manages, and scales your container image. Kubernetes will ensure your system matches this configuration. Three replicas will be created by default.

Deployment name *

agencydb

Namespace *

default

Labels

Use Kubernetes labels to control how workloads are scheduled to your nodes. Labels are applied to all nodes in this node pool.

Key 1 *

app

Value 1

agencydb

[+ ADD KUBERNETES LABEL](#)

Configuration YAML

Kubernetes deployments are defined declaratively using YAML files. The best practice is to store these files in version control, so you can track changes to your deployment configuration over time.

[VIEW YAML](#)

Cluster

Kubernetes Cluster

mysql2 (us-central1)

Cluster in which the deployment will be created.

[CREATE NEW CLUSTER](#)

[CONTINUE](#)

Hình 46: Hình ảnh demo 5

3 Expose (optional)

This operation creates a Kubernetes Service. A service lets your deployment receive traffic and defines how your deployment is exposed. You can always expose your deployment later from the Deployment details page.

☒ Expose deployment as a new service

Port mapping

Item 1 

Port 1

3308

?

Target port 1

3306

?

Protocol 1

TCP

▼

?

+ ADD PORT MAPPING

Service type

Load balancer

▼

?

DEPLOY

Hình 47: Hình ảnh demo 6

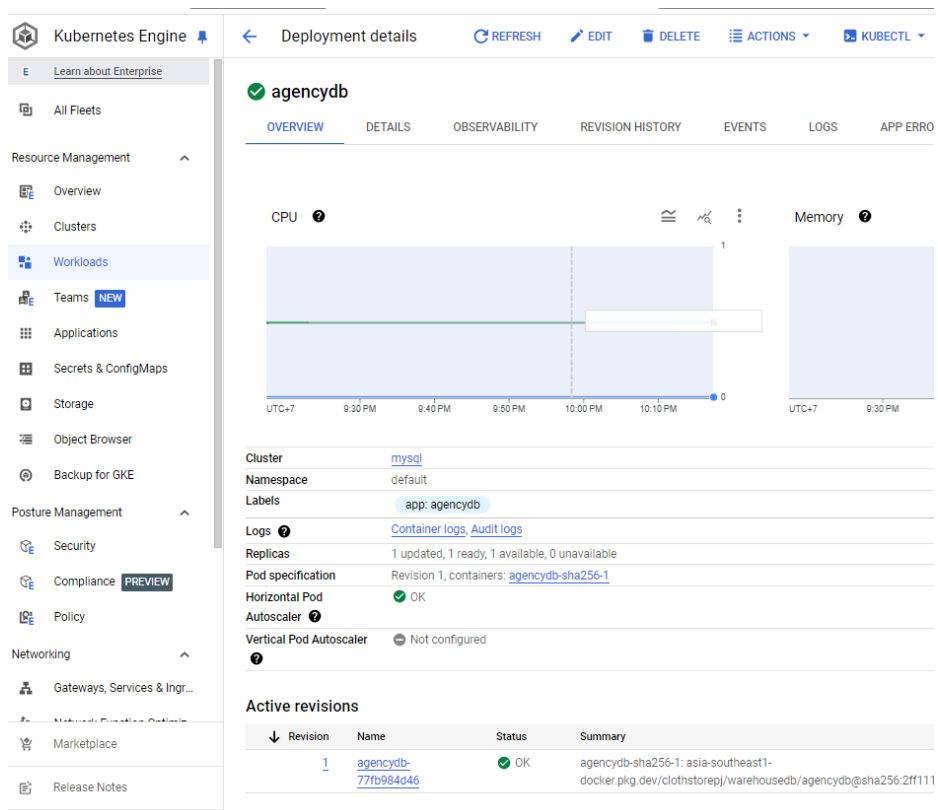
- Tạo file `deployment.yaml` và khai báo deployment cho `agencydb`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: agencydb
spec:
  replicas: 1
  selector:
    matchLabels:
      app: agencydb
  template:
    metadata:
      labels:
        app: agencydb
    spec:
      containers:
      - name: agencydb
        image: gcr.io/YOUR_PROJECT_ID/agencydb:latest
        ports:
        - containerPort: 3306
```

- Áp dụng deployment:

```
kubectl apply -f deployment.yaml
```

Bước 7: Thành công và test connection



Hình 48: hình ảnh demo

Autoscaler

Vertical Pod Autoscaler  Not configured



Active revisions

↓ Revision	Name	Status	Summary
1	agencydb-77fb984d46	 OK	agencydb-sha256-1: asia-docker.pkg.dev/clothstore

Managed pods

Revision	Name	Status	Restarts
1	agencydb-77fb984d46-d6bkr	 Running	0

Exposing services

Name ↑	Type	Endpoints
agencydb-service	Load balancer	34.143.237.175:3308 

Hình 49: hình ảnh demo

- Sau khi deployment hoàn tất, lấy địa chỉ IP và kiểm tra kết nối với MySQL Workbench.

Connection Name:

Connection Remote Management System Profile

Connection Method: Method to use to connect to the RDBMS

Parameters SSL Advanced

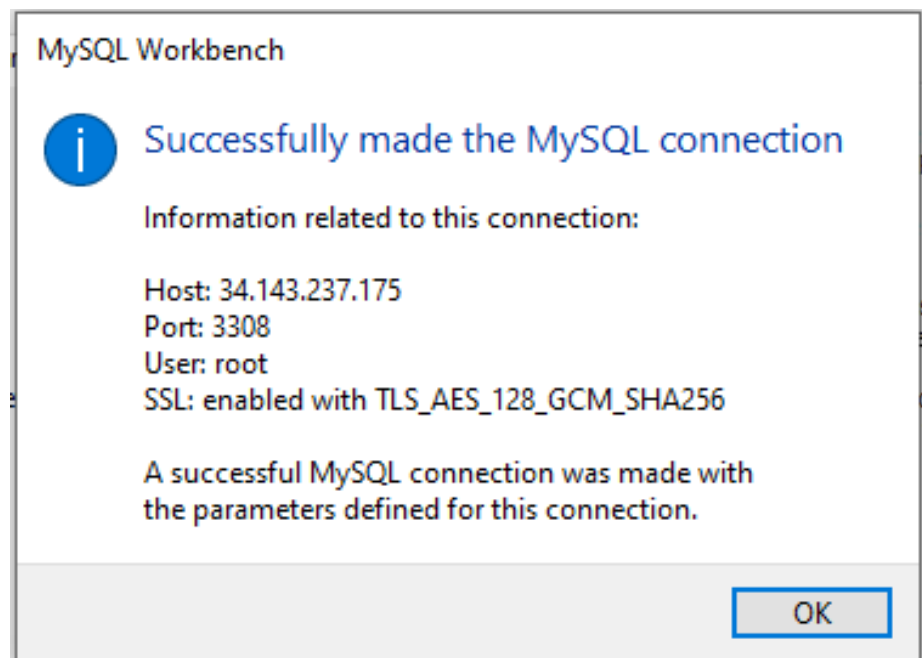
Hostname: Port: Name or IP address of the server host - and TCP/IP port.

Username: Name of the user to connect with.

Password: The user's password. Will be requested later if it's not set.

Default Schema: The schema to use as default schema. Leave blank to select it later.

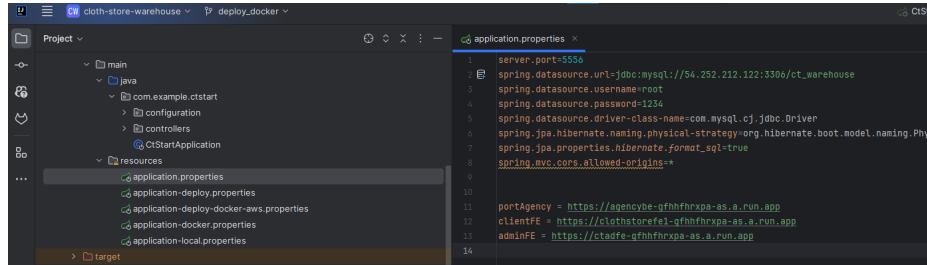
Hình 50: hình ảnh demo



Hình 51: hình ảnh demo

4.7.3 Deploy Docker Image warehousebe và agencybe lên Cloud Run của GCloud bằng Cloud Shell (ví dụ Warehousebe)

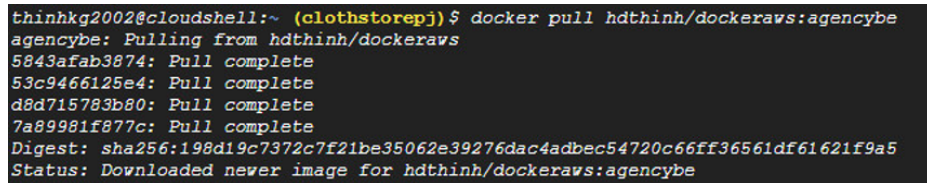
Bước 8: Thay đổi địa chỉ MySQL trong properties thành địa chỉ IP mới



Hình 52: hình ảnh demo

- Mở file cấu hình của ứng dụng và cập nhật địa chỉ IP của MySQL.

Bước 9: Dùng Cloud Shell để pull image warehousedb

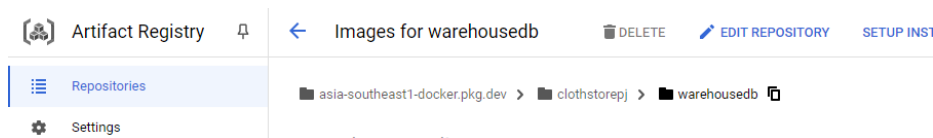


Hình 53: hình ảnh demo

- Pull image từ Docker Hub:

```
docker pull hdthinh/clothstorepj:warehousedb
```

Bước 10: Biến đổi lại tên image bằng Tag



Hình 54: hình ảnh demo

- Tag image với tên mới:

```
docker tag hdthinh/clothstorepj:warehousedb gcr.io/YOUR_PROJECT_ID/warehousedb:latest
```

```
chinhkg2002@cloudshell:~$ docker tag hdthinh/dockeraws:ctadfe asia-southeast1-docker.pkg.dev/clothstorepj/warehouseb/ctadfe:ctadfe
chinhkg2002@cloudshell:~$ docker tag hdthinh/dockeraws:agencybe asia-southeast1-docker.pkg.dev/clothstorepj/warehouseb/agencybe:agencybe
```

Hình 55: hình ảnh demo

Bước 11: Push image vừa tag lên Artifact Registry

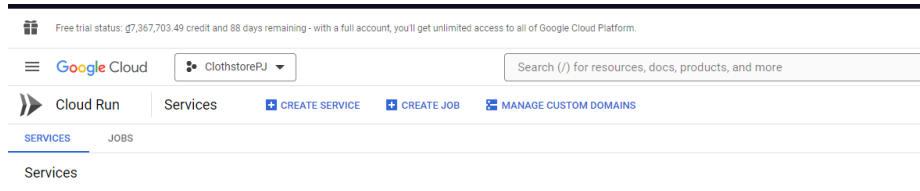
```
chinhkg2002@cloudshell:~ (clothestorepj) $ docker push asia-southeast1-docker.pkg.dev/clothestorepj/warehousedb/agencybe:agencybe
The push refers to repository [asia-southeast1-docker.pkg.dev/clothestorepj/warehousedb/agencybe]
f2f7dc7efc42: Pushed
34f7184834b2: Layer already exists
5836ece05bfd: Layer already exists
72e830a4dff5: Layer already exists
agencybe: digest: sha256:198d19c7372c7f21be35062e39276dac4adbcc54720c66ff36561df61621f9a5 size: 1163
```

Hình 56: hình ảnh demo

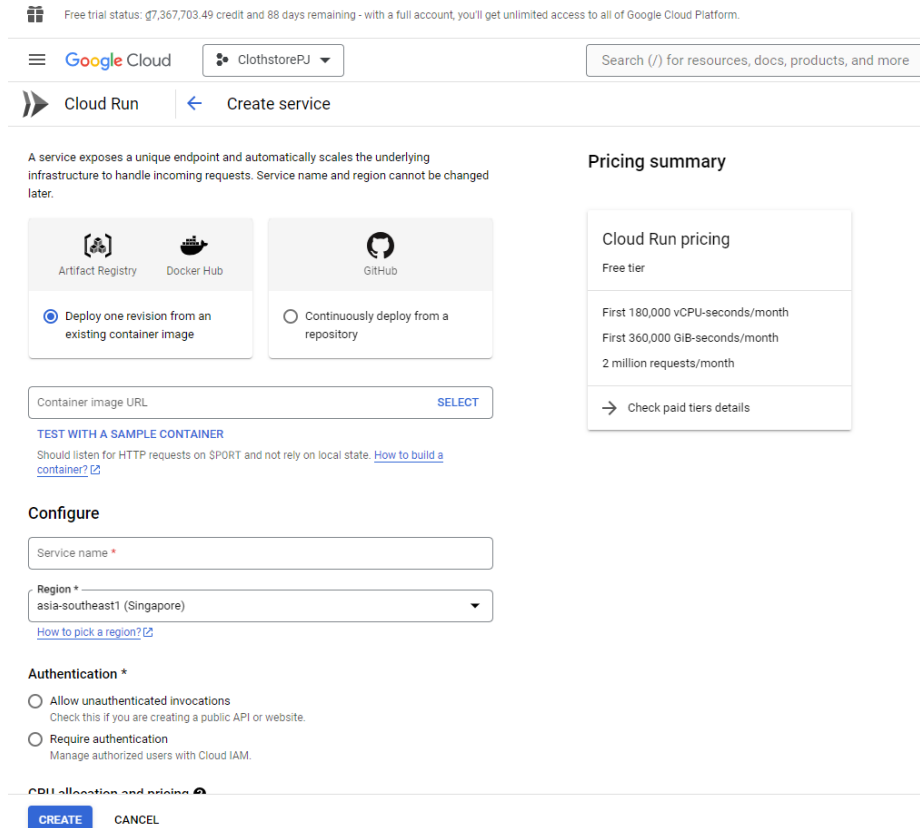
- Push image lên Artifact Registry:

```
docker push gcr.io/YOUR_PROJECT_ID/warehousedb:latest
```

Bước 12: Tạo service trên Cloud Run



Hình 57: hình ảnh demo



Hình 58: hình ảnh demo

- Mở Google Cloud Console, chọn Cloud Run và nhấn **Create Service**.
- Chọn image từ Artifact Registry và cấu hình các tùy chọn cần thiết.

Select container image

ARTIFACT REGISTRY

CONTAINER REGISTRY

Project: clothstorepj

CHANGE

▶ Demo containers

▼ asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb

▶ agencybe

▶ clothstorefe

▶ ctadfe

▶ warehousebe

▼ warehousedb

2bea9c1456

warehousedb

2 days ago

SELECT

CANCEL

Hình 59: hình ảnh demo

Container(s), Volumes, Networking, Security



CONTAINER(S)

VOLUMES

NETWORKING

SECURITY

Edit Container



Container image URL

Not selected

Container port

3306

Requests will be sent to the container on this port. We recommend listening on \$PORT instead of this specific number.

SETTINGS

VARIABLES & SECRETS

VOLUME MOUNTS

Container name is generated by default [EDIT](#)

Container command

Leave blank to use the entry point command defined in the container image.

Container arguments

Arguments passed to the entry point command.

Resources

Memory

512 MiB



Memory to allocate to each instance of this container.

CPU

1



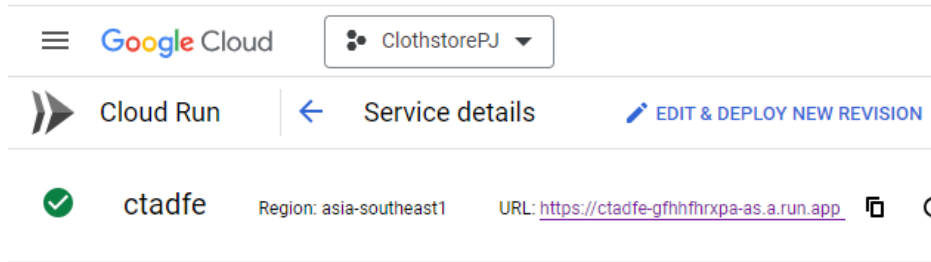
Number of vCPUs allocated to each instance of this container.

CREATE

CANCEL

Hình 60: hình ảnh demo

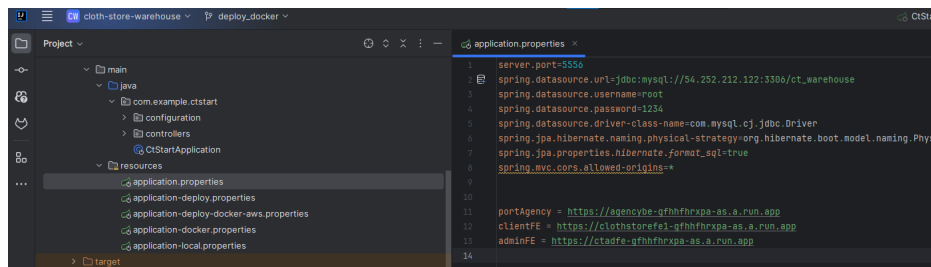
Bước 13: Chờ service hoàn thành và nhận domain



Hình 61: hình ảnh demo

- Sau khi service tạo xong, bạn sẽ nhận được một domain.
- Kiểm tra cấu trúc tên miền của Cloud Run để sử dụng đúng.

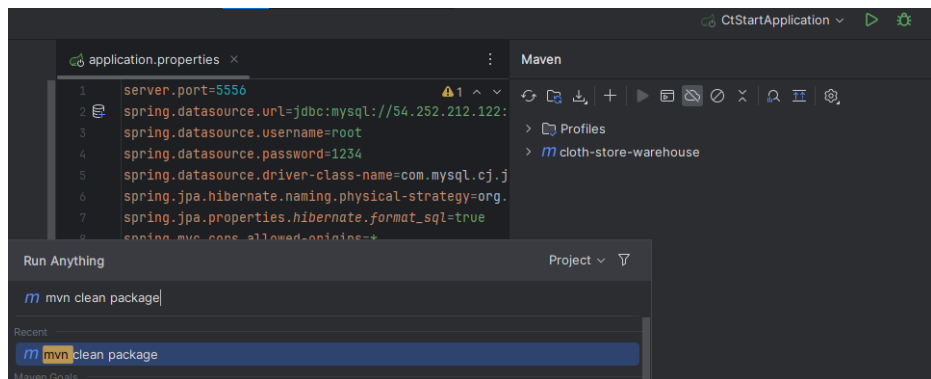
Bước 14: Thay đổi các thành phần trong properties theo domain của GCloud



Hình 62: hình ảnh demo

- Cập nhật cấu hình ứng dụng với domain mới từ Cloud Run.

Bước 15: Đóng gói Project



Hình 63: hình ảnh demo

- Build ứng dụng bằng Maven:

`mvn clean package`

Bước 16: Build image warehousebe

- Build Docker image cho warehousebe:

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\thinh\Desktop\project\cloth-store-warehouse\ct-start> wsl
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.146.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s

```

Hình 64: hình ảnh demo

```

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/thinh/.hushlogin file.
thinh@DESKTOP-AJ5SH34:/mnt/c/Users/thinh/Desktop/project/cloth-store-warehouse/ct-start$ docker build -t hdthinh/dockeraws:warehousebe .

```

Hình 65: hình ảnh demo

```
docker build -t warehousebe .
```

Bước 17: Push image lên repository đã tạo trong Docker Hub

```

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/thinh/.hushlogin file.
thinh@DESKTOP-AJ5SH34:/mnt/c/Users/thinh/Desktop/project/cloth-store-warehouse/ct-start$ docker push hdthinh/dockeraws:warehousebe

```

Hình 66: hình ảnh demo

– Tag và push image lên Docker Hub:

```

docker tag warehousebe hdthinh/clothstorepj:warehousebe
docker push hdthinh/clothstorepj:warehousebe

```


Bước 18: Dùng Cloud Shell để pull image warehousebe

```
thinhkg2002@cloudshell:~ (clothstorepj)$ docker pull hdthinh/dockeraws:agencybe
agencybe: Pulling from hdthinh/dockeraws
5843afab3874: Pull complete
53c9466125e4: Pull complete
d8d715783b80: Pull complete
7a89981f877c: Pull complete
Digest: sha256:198d19c7372c7f21be35062e39276dac4adbec54720c66ff36561df61621f9a5
Status: Downloaded newer image for hdthinh/dockeraws:agencybe
```

Hình 67: hình ảnh demo

- Pull image từ Docker Hub:

```
docker pull hdthinh/clothstorepj:warehousebe
```

Bước 19: Biến đổi lại tên image bằng Tag

```
thinhkg2002@cloudshell:~ (clothstorepj)$ docker tag hdthinh/dockeraws:ctadfe asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/ctadfe:ctadfe
thinhkg2002@cloudshell:~ (clothstorepj)$ docker tag hdthinh/dockeraws:agencybe asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/agencybe:agencybe
```

Hình 68: hình ảnh demo

- Tag image với tên mới:

```
docker tag hdthinh/clothstorepj:warehousebe gcr.io/YOUR_PROJECT_ID/warehousebe:latest
```

Bước 20: Push image vừa tag lên Artifact Registry

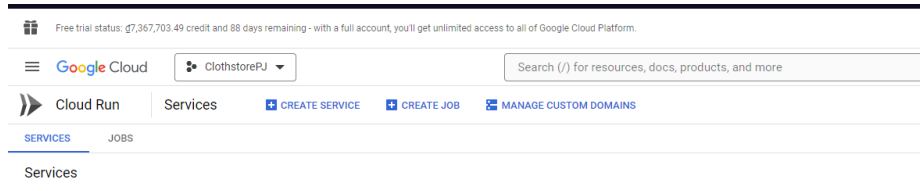
```
thinhkg2002@cloudshell:~ (clothstorepj)$ docker push asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/agencybe:agencybe
The push refers to repository [asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/agencybe]
f2f7dc7efc42: Pushed
34f7184834b2: Layer already exists
5936ece05b4d: Layer already exists
72e830a4dff5: Layer already exists
agencybe: digest: sha256:198d19c7372c7f21be35062e39276dac4adbec54720c66ff36561df61621f9a5 size: 1163
```

Hình 69: hình ảnh demo

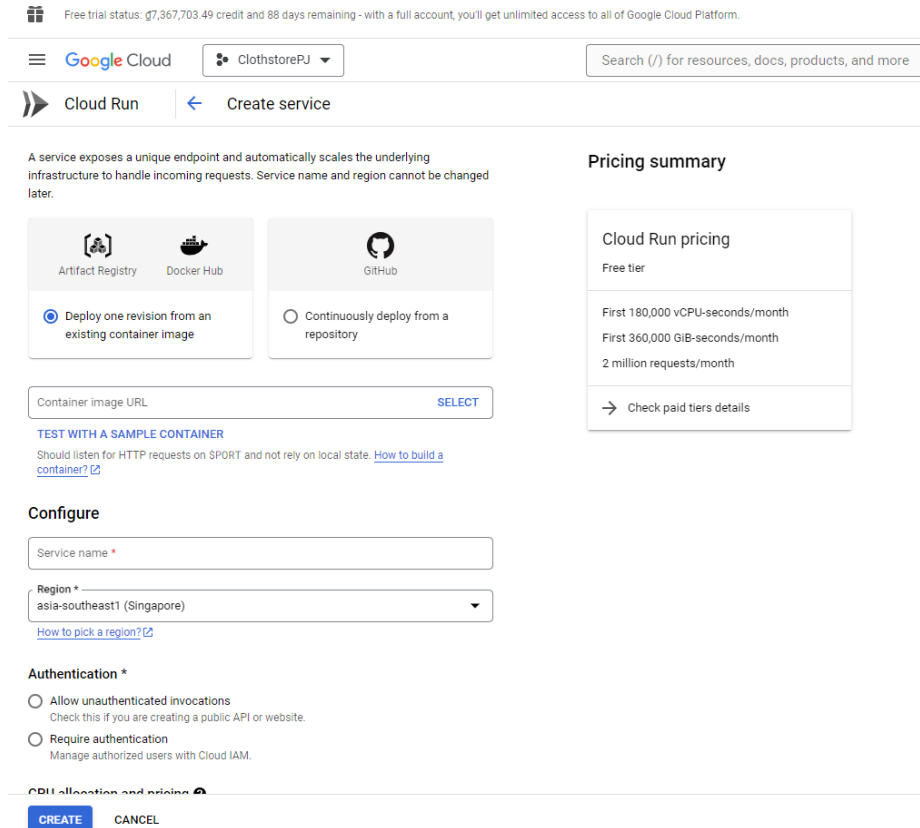
- Push image lên Artifact Registry:

```
docker push gcr.io/YOUR_PROJECT_ID/warehousebe:latest
```

Bước 21: Tạo service trên Cloud Run



Hình 70: hình ảnh demo



Hình 71: hình ảnh demo

- Mở Google Cloud Console, chọn Cloud Run và nhấn **Create Service**.
- Chọn image từ Artifact Registry và cấu hình các tùy chọn cần thiết.

Select container image

ARTIFACT REGISTRY

CONTAINER REGISTRY

Project: clothstorepj

CHANGE

▶ Demo containers

▼ asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb

▶ agencybe

▶ clothstorefe

▶ ctadfe

▶ warehousebe

▼ warehousedb

2bea9c1456

warehousedb

2 days ago

SELECT

CANCEL

Hình 72: hình ảnh demo

Container(s), Volumes, Networking, Security



CONTAINER(S)

VOLUMES

NETWORKING

SECURITY

Edit Container



Container image URL

Not selected

Container port

3306

Requests will be sent to the container on this port. We recommend listening on \$PORT instead of this specific number.

SETTINGS

VARIABLES & SECRETS

VOLUME MOUNTS

Container name is generated by default [EDIT](#)

Container command

Leave blank to use the entry point command defined in the container image.

Container arguments

Arguments passed to the entry point command.

Resources

Memory

512 MiB



Memory to allocate to each instance of this container.

CPU

1



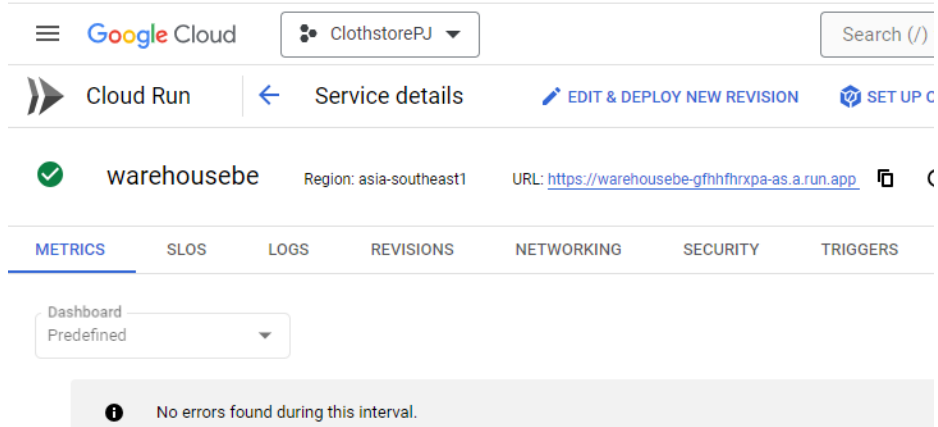
Number of vCPUs allocated to each instance of this container.

CREATE

CANCEL

Hình 73: hình ảnh demo

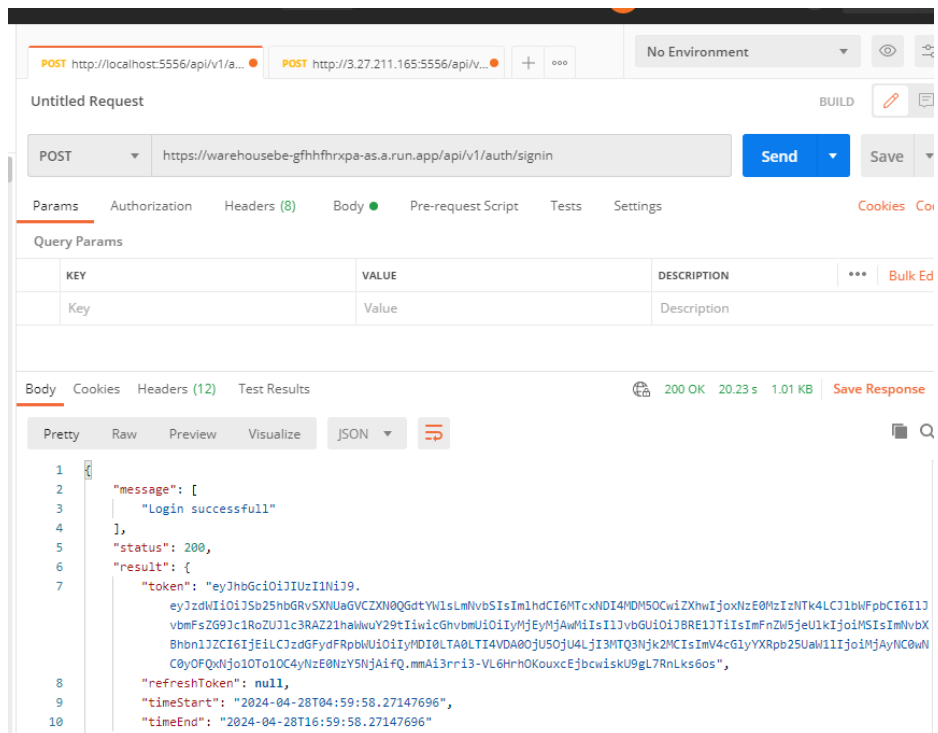
Bước 22: Chờ service hoàn thành và nhận domain



Hình 74: hình ảnh demo

- Sau khi service tạo xong, bạn sẽ nhận được một domain.

Bước 23: Test Postman cho domain mới được tạo



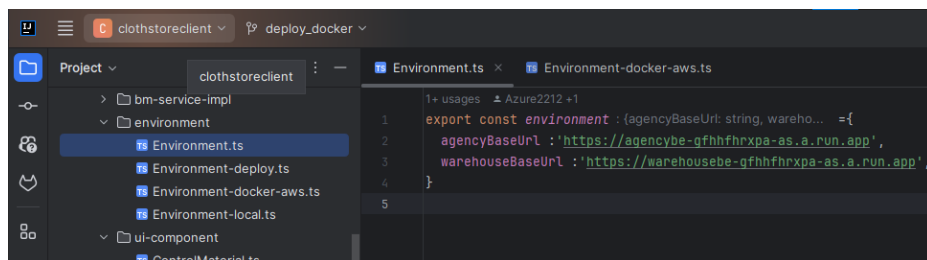
Hình 75: hình ảnh demo

- Sử dụng Postman để kiểm tra API trên domain mới.

4.7.4 Deploy Docker Image clothstorefe và ctadfe lên Cloud Run của GCloud bằng Cloud Shell (ví dụ clothstorefe)

Bước 24: Cập nhật environment để sửa domain cho BE

- Sau khi bước 23 hoàn thành, vào environment file và cập nhật domain của backend services.



Hình 76: hình ảnh demo

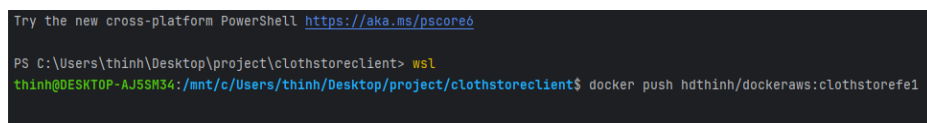
Bước 25: Build image clothstorefe



Hình 77: hình ảnh demo

- Build Docker image cho clothstorefe:
`docker build -t clothstorefe .`

Bước 26: Push image lên repository đã tạo trong Docker Hub



Hình 78: hình ảnh demo

- Tag và push image lên Docker Hub:
`docker tag clothstorefe hdthinh/clothstorepj:clothstorefe`
`docker push hdthinh/clothstorepj:clothstorefe`

Bước 27: Dùng Cloud Shell để pull image clothstorefe

```
thinhkg2002@cloudshell:~ (clothstorepj)$ docker pull hdthinh/dockeraws:agencybe
agencybe: Pulling from hdthinh/dockeraws
5843afab3874: Pull complete
53c9466125e4: Pull complete
d8d715783b80: Pull complete
7a89981f877c: Pull complete
Digest: sha256:198d19c7372c7f21be35062e39276dac4adbec54720c66ff36561df61621f9a5
Status: Downloaded newer image for hdthinh/dockeraws:agencybe
```

Hình 79: hình ảnh demo

- Pull image từ Docker Hub:

```
docker pull hdthinh/clothstorepj:clothstorefe
```

Bước 28: Biến đổi lại tên image bằng Tag

```
thinhkg2002@cloudshell:~ (clothstorepj)$ docker tag hdthinh/dockeraws:ctadfe asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/ctadfe:ctadfe
thinhkg2002@cloudshell:~ (clothstorepj)$ docker tag hdthinh/dockeraws:agencybe asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/agencybe:agencybe
```

Hình 80: hình ảnh demo

- Tag image với tên mới:

```
docker tag hdthinh/clothstorepj:clothstorefe gcr.io/YOUR_PROJECT_ID/clothstorefe:latest
```

Bước 29: Push image vừa tag lên Artifact Registry

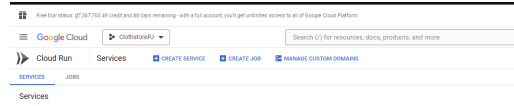
```
thinhkg2002@cloudshell:~ (clothstorepj)$ docker push asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/agencybe:agencybe
The push refers to repository [asia-southeast1-docker.pkg.dev/clothstorepj/warehousedb/agencybe]
c2f7b67ac642: Pushed
c2f7b67ac642: Layer already exists
58f6cc09b6d1: Layer already exists
7a8d9a8f8f81: Layer already exists
agencybe: digest: sha256:198d19c7372c7f21be35062e39276dac4adbec54720c66ff36561df61621f9a5 size: 1143
```

Hình 81: hình ảnh demo

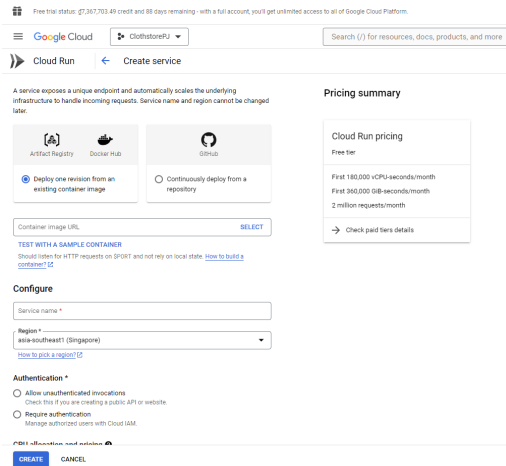
- Push image lên Artifact Registry:

```
docker push gcr.io/YOUR_PROJECT_ID/clothstorefe:latest
```

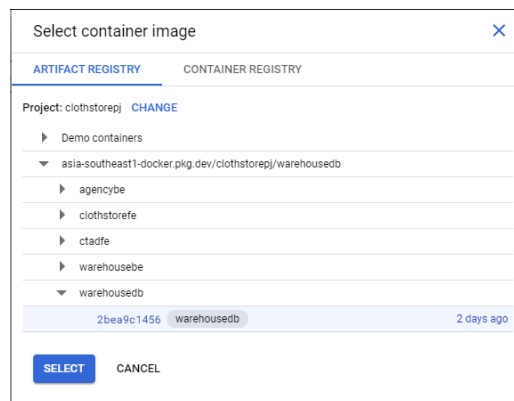
Bước 30: Tạo service trên Cloud Run



Hình 82: hình ảnh demo



Hình 83: hình ảnh demo



Hình 84: hình ảnh demo

- Mở Google Cloud Console, chọn Cloud Run và nhấn **Create Service**.
- Chọn image từ Artifact Registry và cấu hình các tùy chọn cần thiết, bao gồm cổng đã cấu hình trong Docker.

Container(s), Volumes, Networking, Security ^

CONTAINER(S) VOLUMES NETWORKING SECURITY

Edit Container  ^

Container image URL
Not selected

Container port

Requests will be sent to the container on this port. We recommend listening on \$PORT instead of this specific number.

SETTINGS VARIABLES & SECRETS VOLUME MOUNTS

Container name is generated by default [EDIT](#)

Container command
Leave blank to use the entry point command defined in the container image.

Container arguments
Arguments passed to the entry point command.

Resources

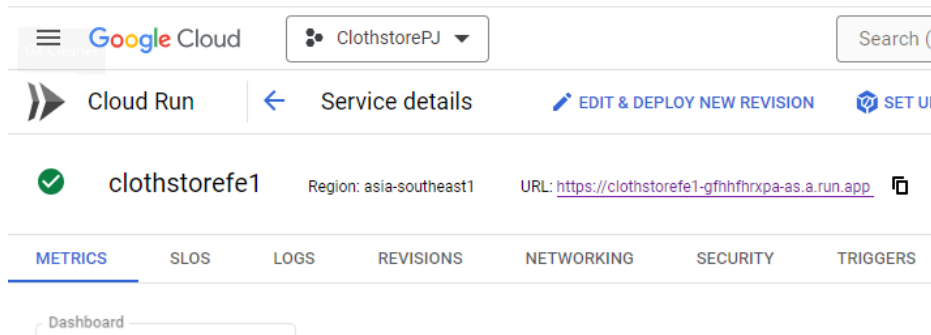
Memory
Memory to allocate to each instance of this container.

CPU
Number of vCPUs allocated to each instance of this container.

CREATE CANCEL

Hình 85: hình ảnh demo

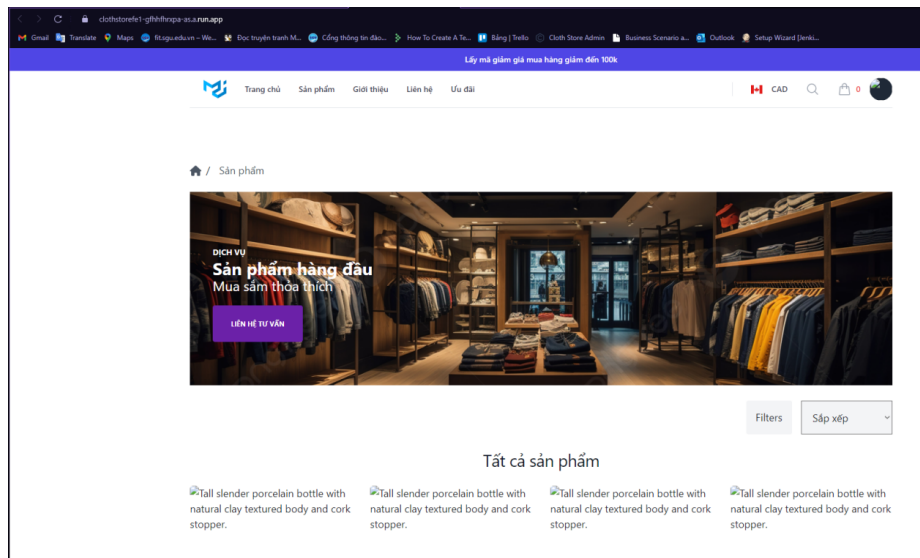
Bước 31: Chờ service hoàn thành và nhận domain



Hình 86: hình ảnh demo

- Sau khi service tạo xong, bạn sẽ nhận được một domain.

Bước 32: Nhấp vào domain vừa được tạo để test



Hình 87: hình ảnh demo

- Mở trình duyệt và kiểm tra domain mới tạo để đảm bảo ứng dụng hoạt động đúng.

5 CONCLUSION

Trong bài giới thiệu này, chúng ta đã khám phá sâu hơn về Git, một hệ thống quản lý phiên bản phân tán mạnh mẽ được sử dụng rộng rãi trong phát triển phần mềm ngày nay.

Git có một kiến trúc hoạt động phân tán, cho phép nhiều người làm việc trên cùng một dự án mà không cần phụ thuộc vào một máy chủ trung tâm. Thay vào đó, mỗi máy tính trong mạng Git đều chứa toàn bộ lịch sử phiên bản của dự án. Điều này tạo điều kiện cho sự phân phối linh hoạt và đồng bộ hóa dữ liệu giữa các thành viên trong nhóm.

Một số cú pháp thông dụng để sử dụng của Git đã được liệt kê sử dụng để tương tác với các kho lưu trữ từ xa như GitHub hoặc GitLab. Ngoài ra để trực quan hóa

các hoạt động của lập trình viên cho việc quản lý mã nguồn bằng Git mà ngay nay các công cụ trực quan hóa Git đã ra đời. Có thể kể đến tiêu biểu nhất là Github Desktop[2], cho phép người dùng thao tác các câu lệnh Git 1 cách dễ hình dung mà không cần sử dụng lệnh

Việc hiểu và sử dụng thành thạo Git là một kỹ năng quan trọng đối với các nhà phát triển phần mềm, giúp họ quản lý hiệu quả mã nguồn và làm việc hiệu quả trong môi trường phát triển phần mềm phức tạp.

Tài liệu

- [1] *Git Tutorial*, [Online].
Available: <https://www.w3schools.com/git/>
- [2] *Github Desktop*, [Online].
Available: <https://docs.github.com/en/desktop/overview/getting-started-with-github-desktop>